

Escola Superior de Tecnologia e Gestão de Águeda

---

# Sistemas Operativos

Técnico Superior Profissional em Redes e Sistemas Informáticos

## Gestão de Memória

**Professor:** Ivan Miguel Pires

2025/2026

# OBJETIVOS E BIBLIOGRAFIA

Descrever as várias formas de **organização** de memória

Discutir várias técnicas de **gestão** de memória

“Operating System Concepts”. Abraham Silberschatz, Peter B. Galvin, Greg Gagne, 9th edition, 2014, Capítulo 8

**NOTA: Os acetatos que se seguem não substituem a bibliografia aqui referida, e deverão por isso ser vistos apenas como um complemento para o estudo da matéria.**

# PLANO DA AULA

## Introdução e enquadramento

## Swapping

## Gestão de memória

- Reserva contígua de memória

- Segmentação

- Paginação

  - Estrutura da tabela de páginas

## Motivação

O objetivo de um computador é executar programas.

Para aumentar a utilização do CPU e melhorar o tempo de resposta para os utilizadores é necessário manter vários programas **em memória**.

**Como fazer a gestão da memória?**



# INTRODUÇÃO

O CPU apenas acede **diretamente** à memória e aos seus registos

logo, para que um programa se possa executar, é preciso que este seja **carregado** para memória (a partir do disco)

Enquanto o acesso aos registos é rápido (1 ciclo de CPU) o acesso à memória é mais lento (vários ciclos)

tipicamente usa-se uma **cache** (memória mais rápida) entre CPU e memória principal

E como se **protege** o sistema?

processo do utilizador não pode Exemplo: aceder a código/dados de outros processos/SO

proteção feita por hardware porque o SO seria demasiado lento:  
por exemplo, usando registo *base* + *limite* definindo o **espaço de endereçamento lógico** do processo

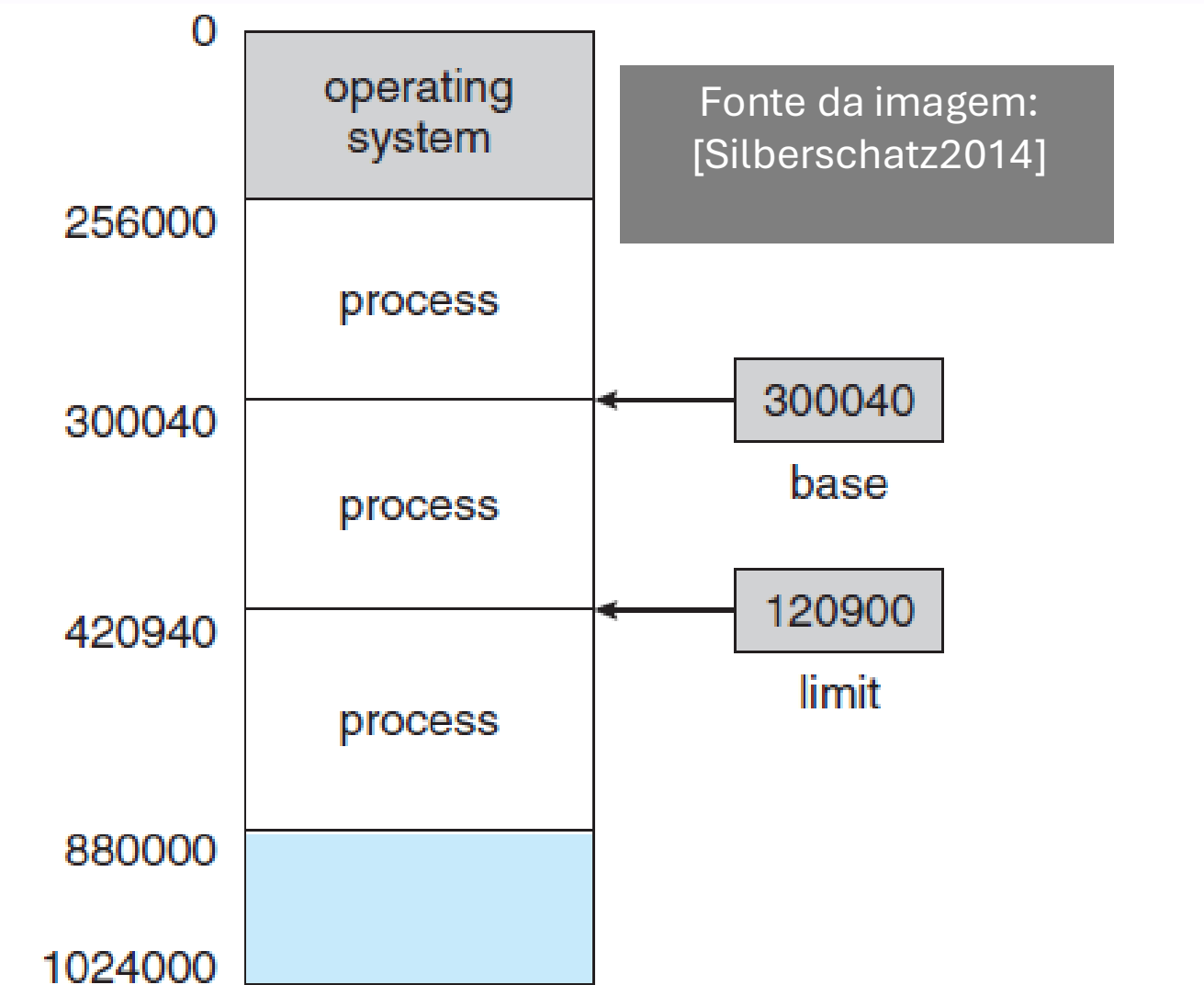


Figure 8.1 A base and a limit register define a logical address space.

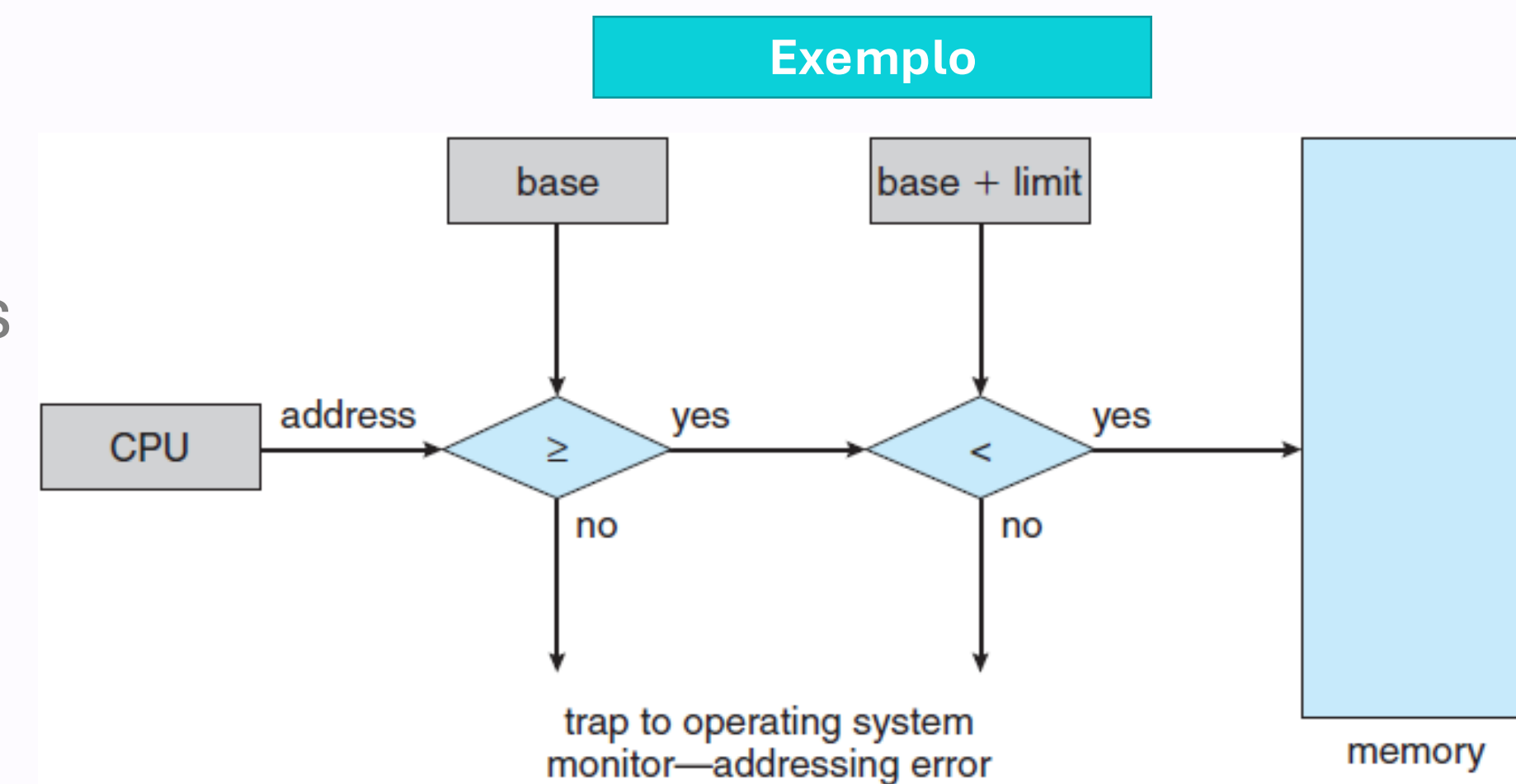


Figure 8.2 Hardware address protection with base and limit registers.

# GESTÃO DA MEMÓRIA - ENDEREÇAMENTO REAL: SISTEMAS MONOPROGRAMADOS

## **Sistemas monoprogramados:**

- Apenas existe um programa em execução que tem acesso total à memória principal (memória física).
- O espaço de endereçamento do processo está limitado ao espaço de endereçamento físico.
- Alocação contígua de memória.

## **Mecanismo de sobreposição(overlay):**

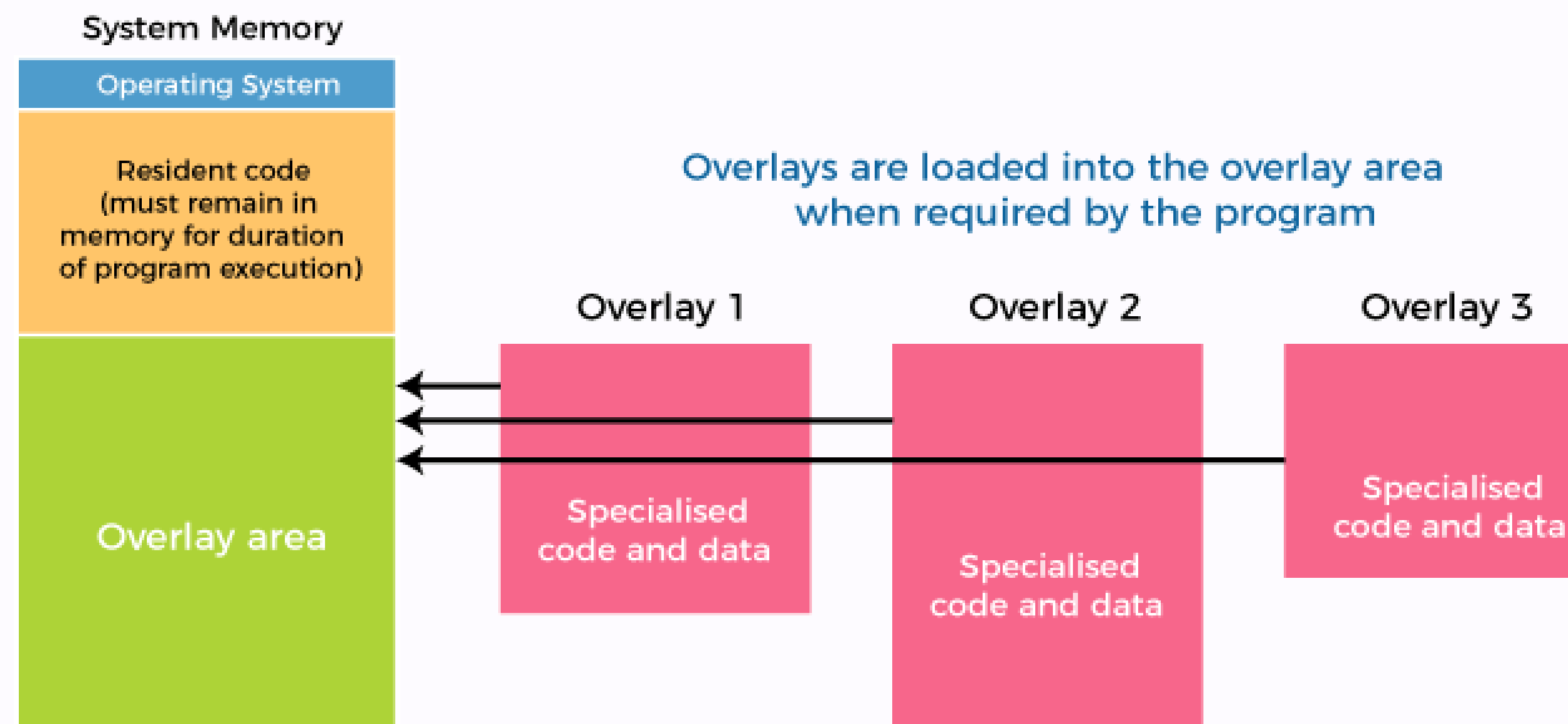
- Permite executar programas com dimensão superior à memória física.
- Continuam a ter de ser dimensionados para a memória física disponível.
- Mantém-se em memória apenas as instruções e os dados que são necessários em determinado momento.

# GESTÃO DA MEMÓRIA - ENDEREÇAMENTO REAL: SISTEMAS MONOPROGRAMADOS

## Mecanismo de sobreposição(overlay):

São geridos pelo utilizador. O programador indica explicitamente quando deve ser carregado um novo overlay. O SO não oferece qualquer suporte a este mecanismo.

Dificuldades: certos programas não são facilmente organizáveis em overlays; pode condicionar o desempenho se as trocas de overlays ocorrerem com muita frequência; dificulta a manutenção e atualização do programa.





# GESTÃO DA MEMÓRIA - ENDEREÇAMENTO REAL: SISTEMAS MULTIPROGRAMADOS

## Partições com dimensão fixa:

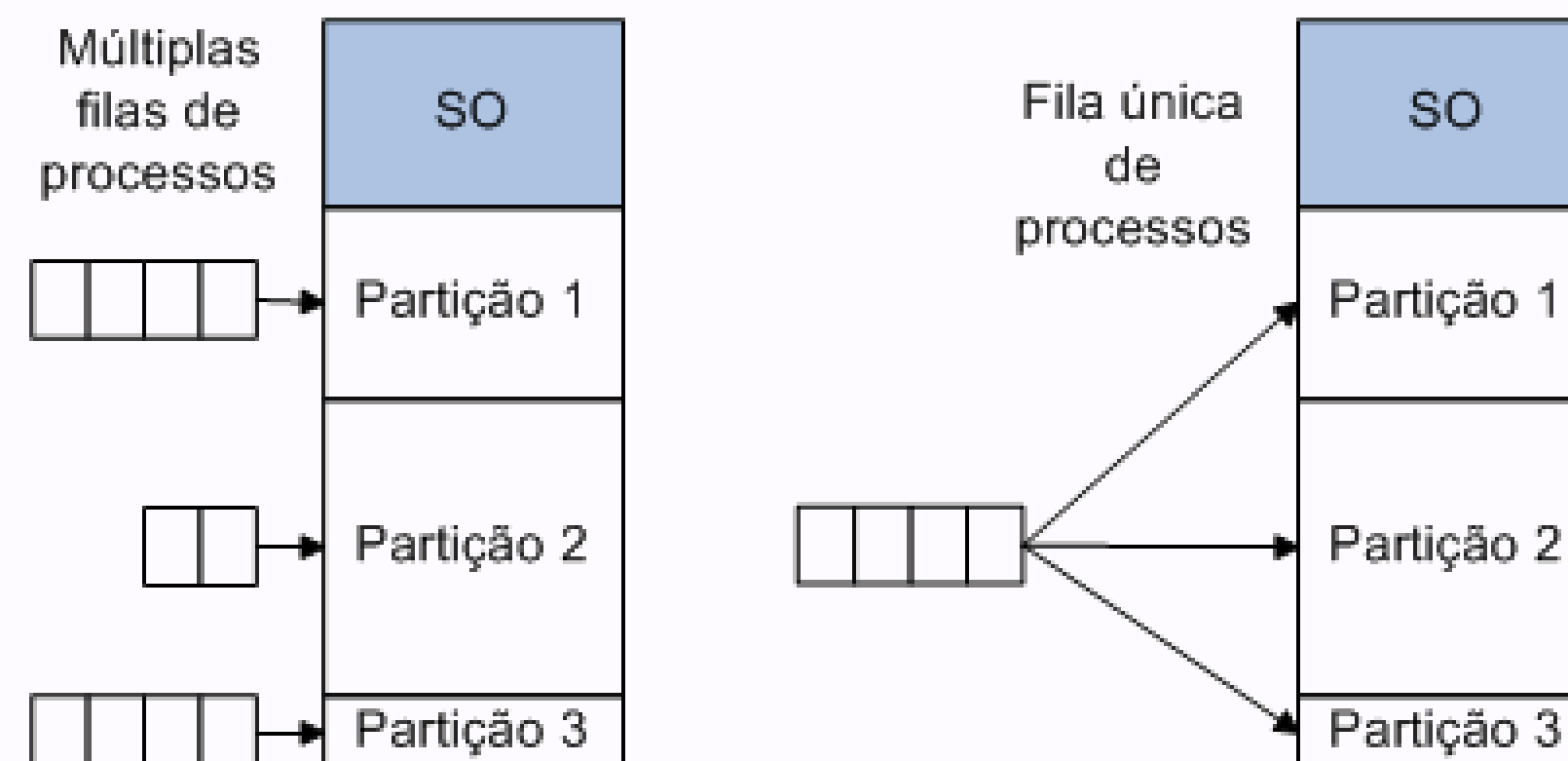
A memória principal é dividida em partições de dimensão fixa.

Para cada partição o SO mantém uma fila de espera dos processos prontos a executar. Sempre que um processo fica bloqueado, o SO faz o despacho do processo em fila de espera associada a outra partição. Em alternativa, poderá ser utilizada apenas uma fila de espera sendo selecionada, aquando do despacho, a partição que melhor se adapta ao processo.

Alocação contígua de memória.

Programas relocáveis com recurso ao endereçamento baseado.

Protecção garantida pelo endereçamento baseado.



# GESTÃO DA MEMÓRIA - ENDEREÇAMENTO REAL: SISTEMAS MULTIPROGRAMADOS

## **Partições com dimensão fixa:**

### **Desvantagens:**

Qual o número de partições e a sua dimensão?

Fragmentação interna: a dimensão dos processos não coincide exactamente com a dimensão da partição.

Poderão existir filas de espera vazias para algumas partições.

Dimensão do programa limitada à dimensão da maior partição, podendo, no entanto, recorrer ao mecanismo de overlays para suprir esta restrição.



# GESTÃO DA MEMÓRIA - ENDEREÇAMENTO REAL: SISTEMAS MULTIPROGRAMADOS

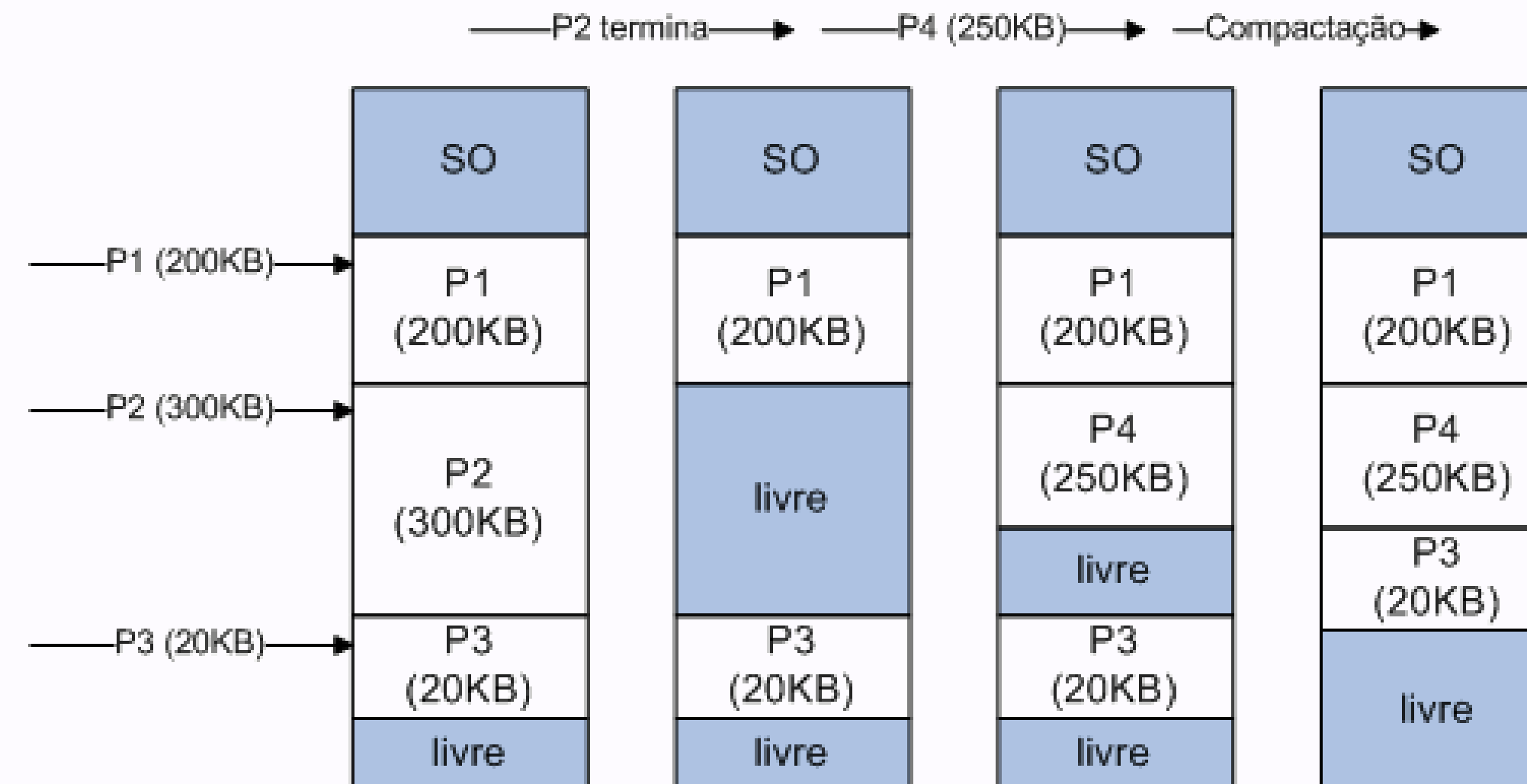
## Partições com dimensão variável:

A dimensão da partição é ajustada à dimensão do programa, evitando, assim, a fragmentação interna. No entanto, à medida que os processos vão terminando, ficam espaços de memória livres (fragmentação externa).

Alocação contígua de memória.

Programas relocáveis com recurso ao endereçamento baseado.

Protecção garantida pelo endereçamento baseado.



# GESTÃO DA MEMÓRIA - ENDEREÇAMENTO REAL: SISTEMAS MULTIPROGRAMADOS

## **Partições com dimensão variável:**

### **Desvantagens:**

Qual o número de partições e a sua dimensão?

Fragmentação interna: a dimensão dos processos não coincide exactamente com a dimensão da partição.

Poderão existir filas de espera vazias para algumas partições.

Dimensão do programa limitada à dimensão da maior partição, podendo, no entanto, recorrer ao mecanismo de overlays para suprir esta restrição.

# GESTÃO DE MEMÓRIA CONTIGUA

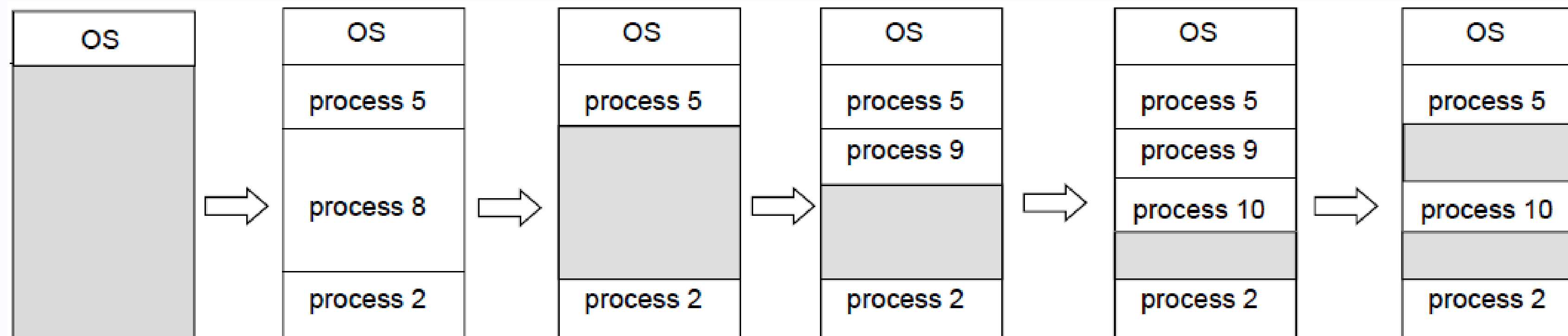
- A memória principal tem habitualmente duas **partições**:
  - o SO residente, normalmente colocado em endereços baixos de memória, para ficar próximo do vetor de interrupções
  - os processos do utilizador
- Quando se usa o mecanismo de reserva contígua de memória cada processo é colocado **numa única secção** de memória, que é **contígua** à secção de memória de outro processo
- A **proteção e mapeamento** são feitos usando:
  - um **registo limite** que define gama de endereços lógicos
  - um **registo base** (de relocação) com o valor do endereço físico mais baixo



# RESERVA CONTÍGUA DE MEMÓRIA

- **SO tem tabela com informação sobre os blocos de memória ocupados e livres**

- no início toda a memória está disponível e é considerada um único bloco livre – um grande “buraco” (hole)
- os processos vão sendo colocados em memória enquanto existirem buracos onde eles “caibam”
- à medida que os processos terminam o seu bloco fica livre, deixando um buraco, em que os buracos adjacentes “unem-se” para formar buracos maiores



- **Algoritmos de reserva de memória**

- First-fit – escolhe o **primeiro** buraco livre (o + rápido)
- Best-fit – escolhe o buraco **menor** com dimensão suficiente
- Worst-fit – escolhe o buraco **maior**

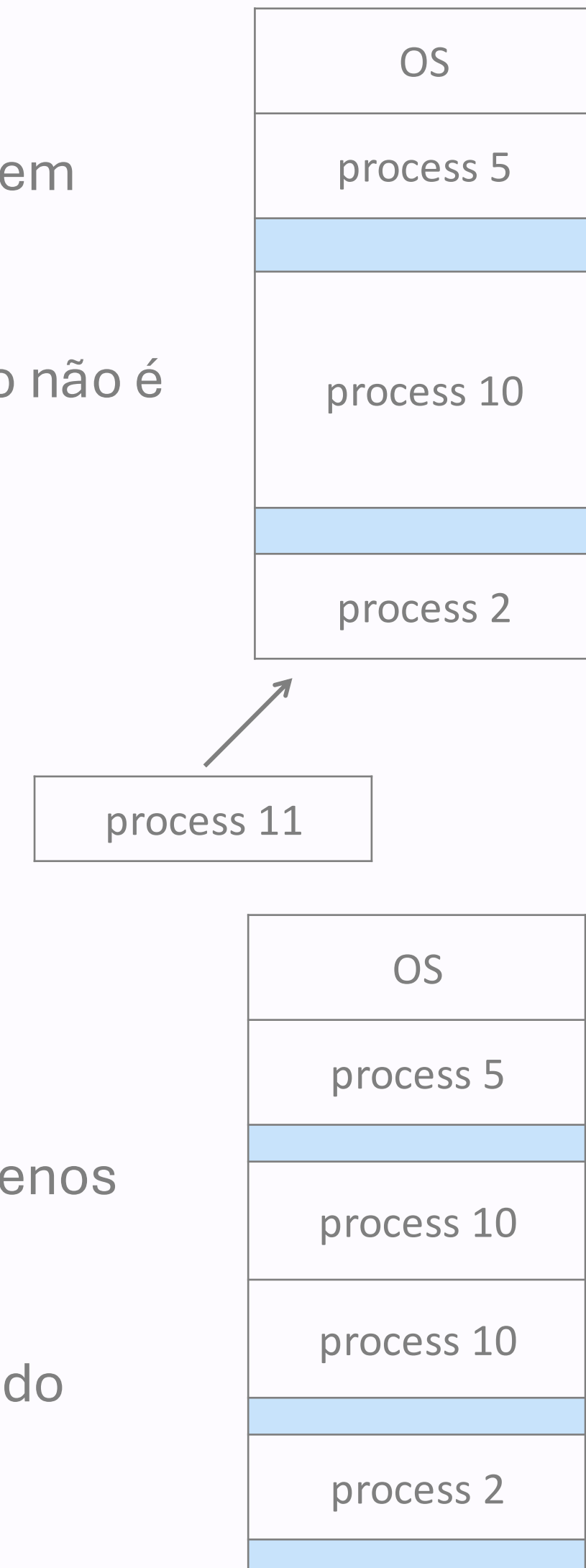
# PROBLEMA: FRAGMENTAÇÃO

## Fragmentação **externa**

- à medida que os processos são carregados e removidos da memória, o espaço livre fica **partido** em buracos pequenos
- pode chegar-se a um ponto em que existe memória livre para satisfazer um pedido **mas** o espaço não é contíguo
- possível solução (muito dispendiosa): **compactação**
  - “baralham-se” os buracos de forma a colocá-los todos juntos, formando um **grande** bloco livre
  - só possível quando a relocação é dinâmica (endereços atribuídos em tempo de execução)
- **Solução melhor**: o espaço de memória de um processo não ter de ser contíguo (paginação segmentação)

## ▪ Fragmentação **interna**

- muitas vezes a memória é dividida em partes de **igual** dimensão, para evitar buracos muito pequenos
- neste caso a reserva é feita em blocos da mesma dimensão
- a memória reservada para um processo pode ser superior à necessária, e o **excedente** não é usado



# ENDEREÇOS LÓGICOS VS FÍSICOS

O conceito de espaço de endereçamento **lógico** ligado a um espaço separado de endereçamento **físico** é central na gestão de memória

Endereço lógico (ou virtual) é aquele gerado pelo CPU; o endereço físico é aquele que é visto pela unidade de memória

endereços lógico e físico são **idênticos** se a atribuição de endereços for feita em tempo de compilação ou de carregamento...

... mas são **diferentes** se for feita em tempo de execução

A MMU (**Memory-Management Unit**) é o hardware que trata dessa **conversão**:

endereço lógico → endereço físico

os programas de utilizador só lidam com endereços lógicos

## Exemplo

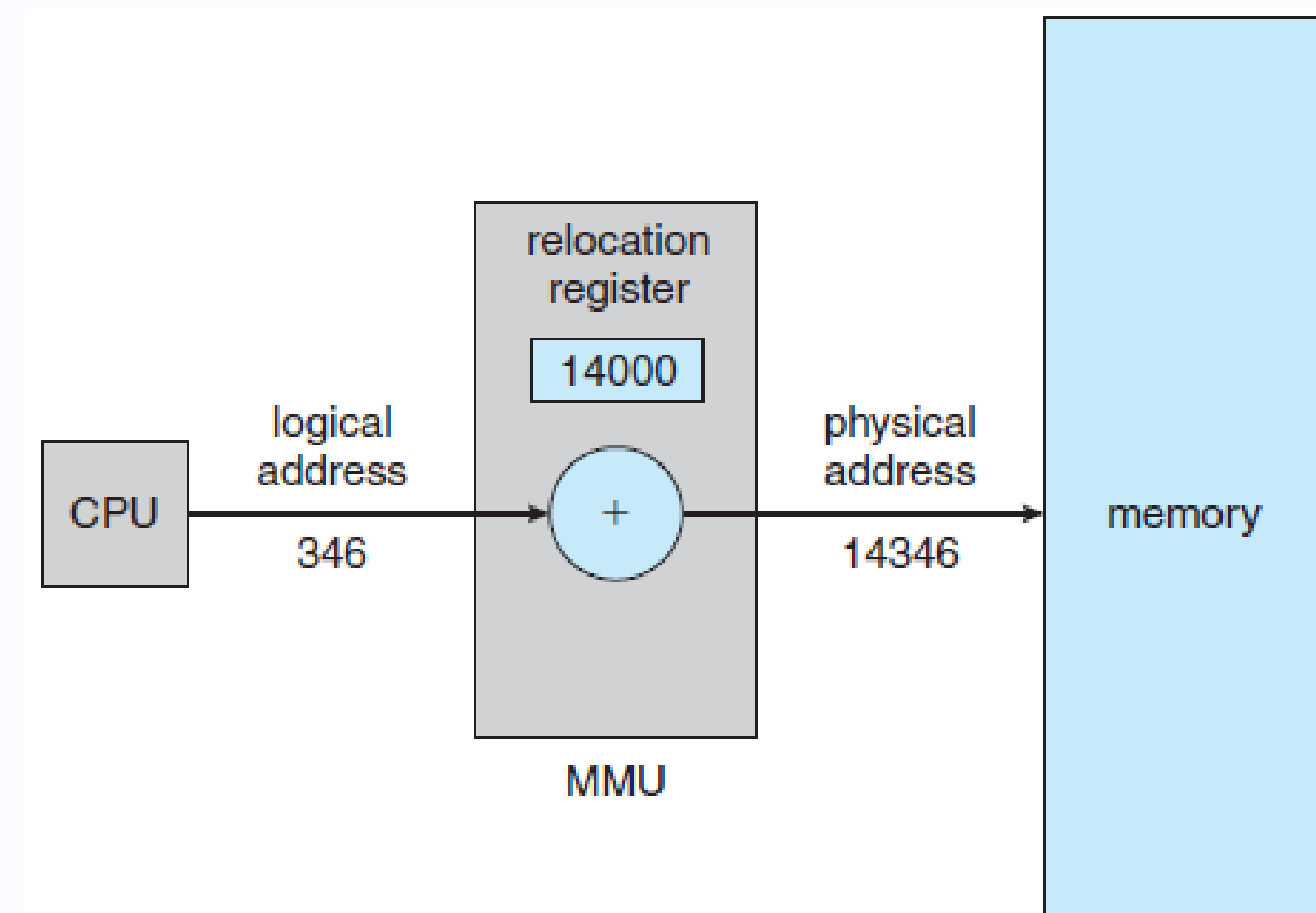
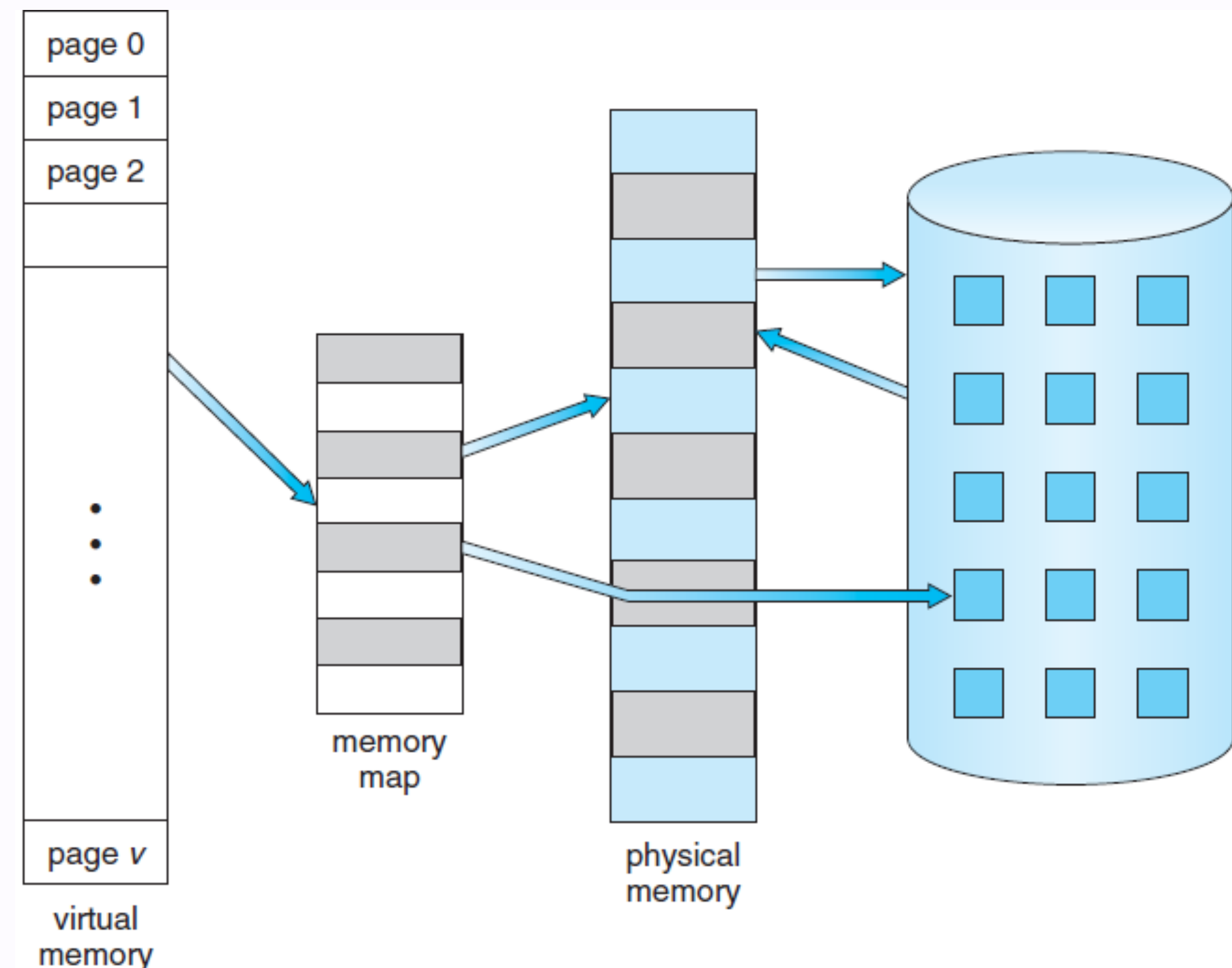


Figure 8.4 Dynamic relocation using a relocation register.



# GESTÃO DA MEMÓRIA - ENDEREÇAMENTO VIRTUAL

- Apenas são mantidos em memória física os blocos necessários a cada processo. Os restantes blocos são mantidos em memória secundária (swapping).
- Permite a alocação não contígua de memória.





# GESTÃO DA MEMÓRIA - ENDEREÇAMENTO VIRTUAL

Separação de memória lógica da memória física

O espaço de endereçamento lógico dum processo **pode ser maior do que a memória física**

A soma dos espaços de endereçamento de todos os processos pode ultrapassar a memória física.

Apenas a parte “ativa” (working set) do programa precisa de estar em memória – mais eficaz!

Fornece mecanismos de simplificação de gestão de memória e proteção

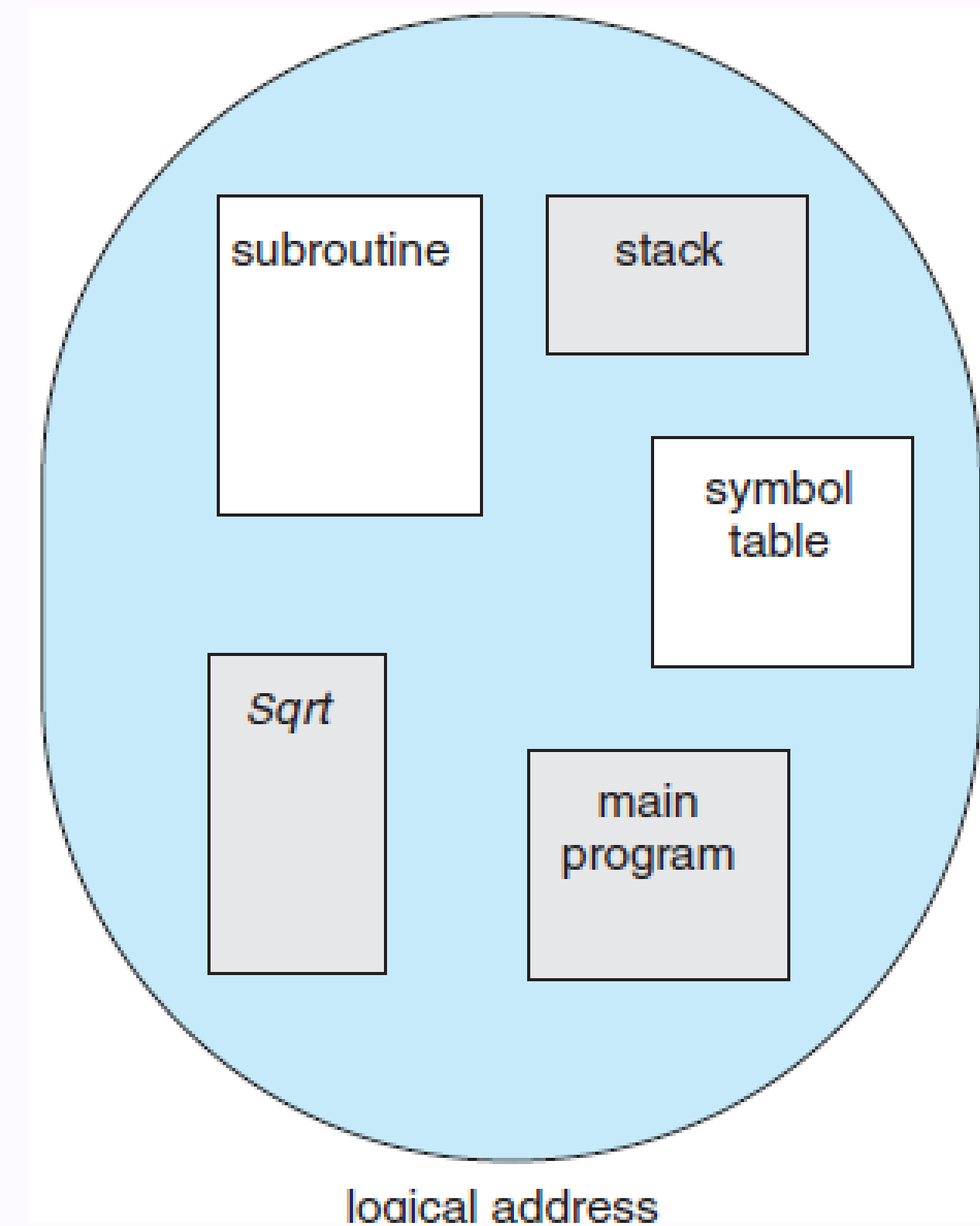
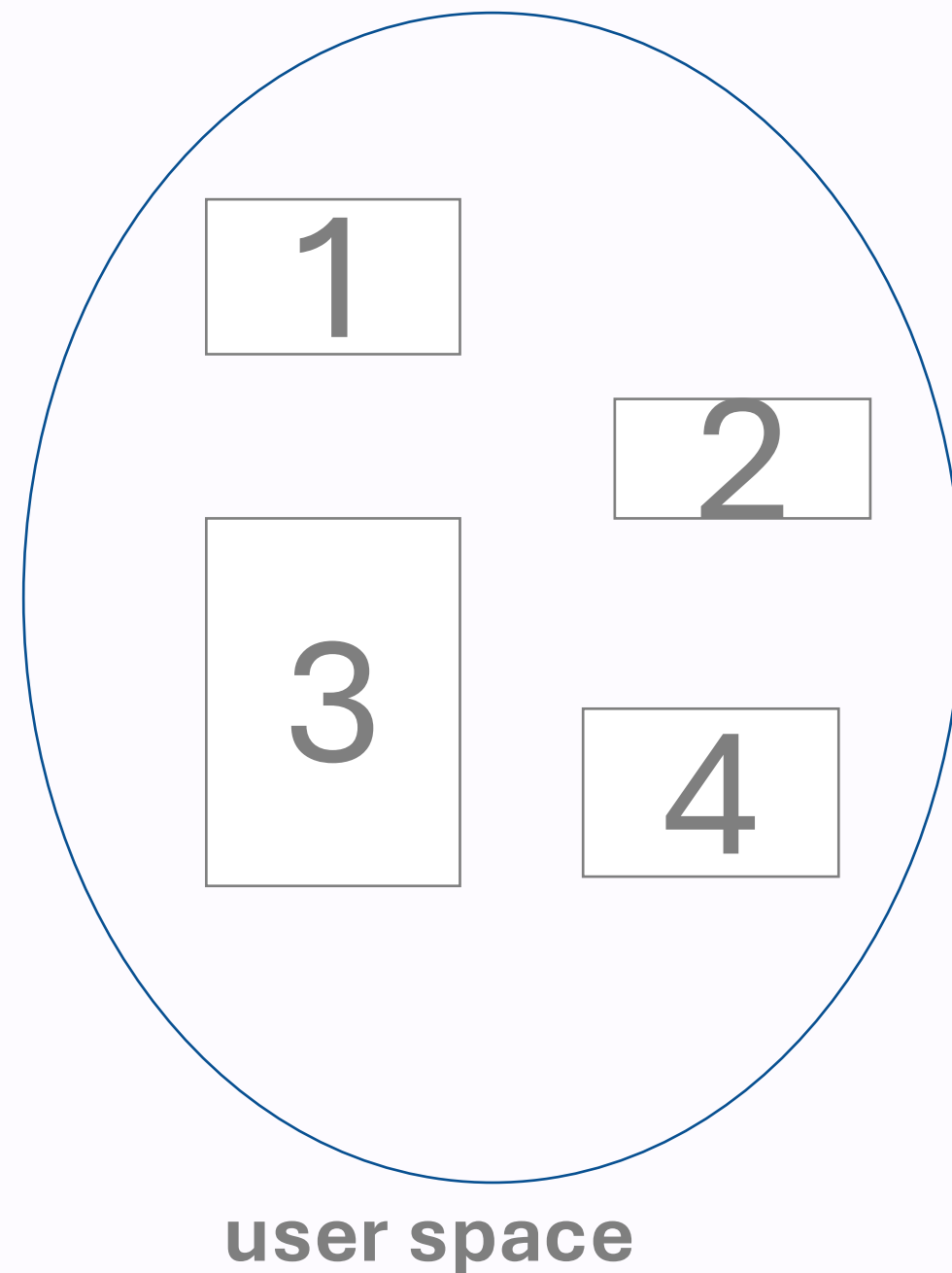
## Implementações

- Demand paging

- Demand Segmentation

# SEGMENTAÇÃO

- Esquema de gestão de memória que replica a forma como o **utilizador** vê a memória
  - para o utilizador um programa é um conjunto de **segmentos** de tamanho



O endereço lógico consiste em:

**<ID segmento, offset>**

# SUPORTE DE HARDWARE PARA SEGMENTAÇÃO

- Endereço virtual: (segmento, deslocamento)

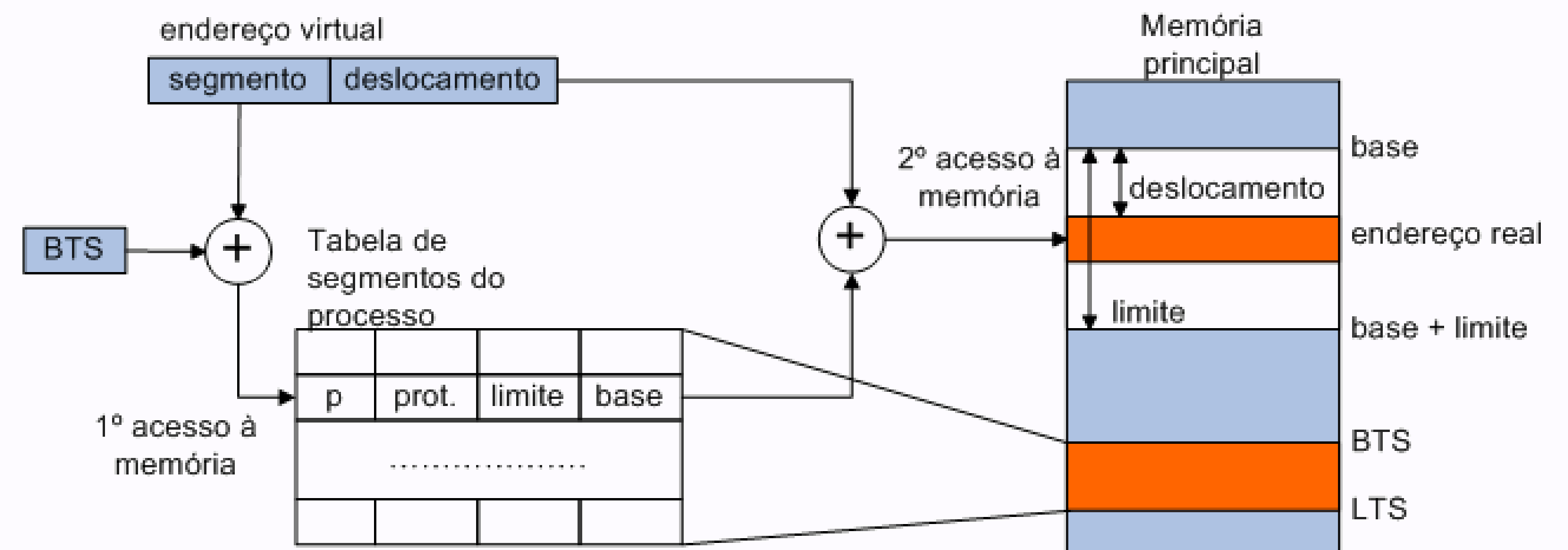
- Determinação do endereço real (MMU)

- Tabela de segmentos:**

- mapeia endereços lógicos (2D) em endereços físicos (1D)

- armazenada na memória principal entre o endereço BTS (base da tabela de segmentos) e LTS (limite da tabela de segmentos). O bit p indica se o segmento está presente na memória principal. Se não estiver, é gerada uma exceção (falta de segmento) que permitirá ao SO carregar o segmento da memória secundária para a memória principal.

- accedida apenas a primeira vez em que um segmento é necessário, em acessos subsequentes, o endereço base é guardado em registos internos à CPU (ex. Intel x86 registos DS, CS, SS, ES).



- Cada entrada da tabela contém:

- base – **endereço inicial** da memória física onde o segmento reside
    - limit – tamanho do segmento



# GESTÃO DA MEMÓRIA - ENDEREÇAMENTO VIRTUAL: SEGMENTAÇÃO

## **Proteção:**

Processos diferentes têm tabelas de segmentos diferentes. Os espaços de endereçamento virtual são distintos (inacessível a outros processos);

Verificação do limite de endereçamento dentro de cada um dos segmentos, comparando o deslocamento com esse limite;

Verificação do tipo de acesso ao segmento: leitura, escrita e execução;

## **Partilha de memória entre processos:**

basta colocar nas tabelas de segmentos dos processos o endereço real do segmento a partilhar;

Os endereços virtuais usados para aceder ao segmento partilhado podem ser diferentes nos vários processos;

A protecção de um segmento partilhado é definida para cada processo através da respetiva tabela de segmentos.

Os segmentos têm tamanhos variáveis pelo que os algoritmos de alocação da memória são complicados.

Fragmentação externa: devido à alocação e libertação de espaços de dimensão variável.

Os programadores têm de ter uma noção mínima do modelo de endereçamento.

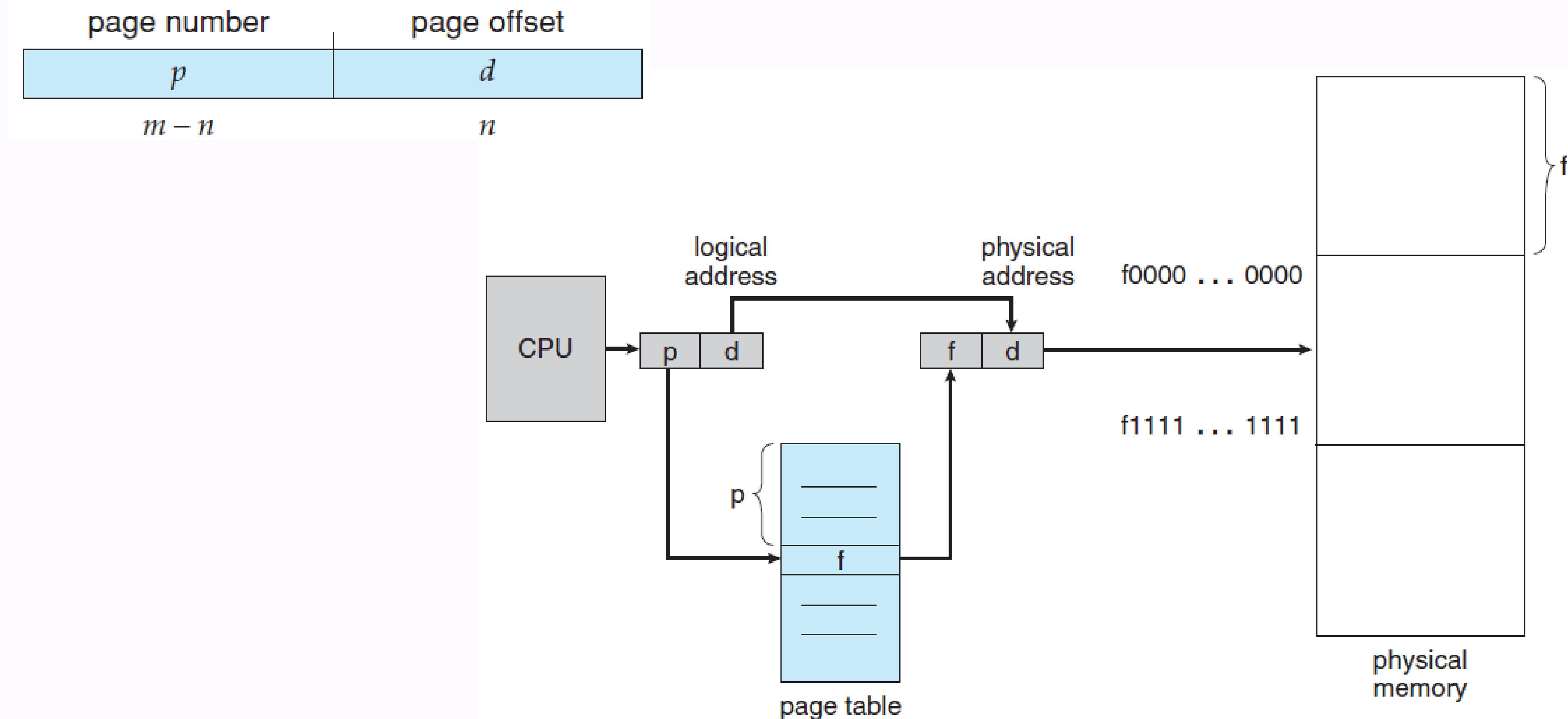
Partilha de código e dados entre utilizadores facilitada.

Crescimento dinâmico de segmentos facilitado (alargamento do espaço de endereçamento).

A protecção de código e dados pode ser realizada separadamente.

# SUPORTE DE HARDWARE PARA SEGMENTAÇÃO

- O endereço gerado pelo CPU é **dividido** em duas partes:



# PAGINAÇÃO

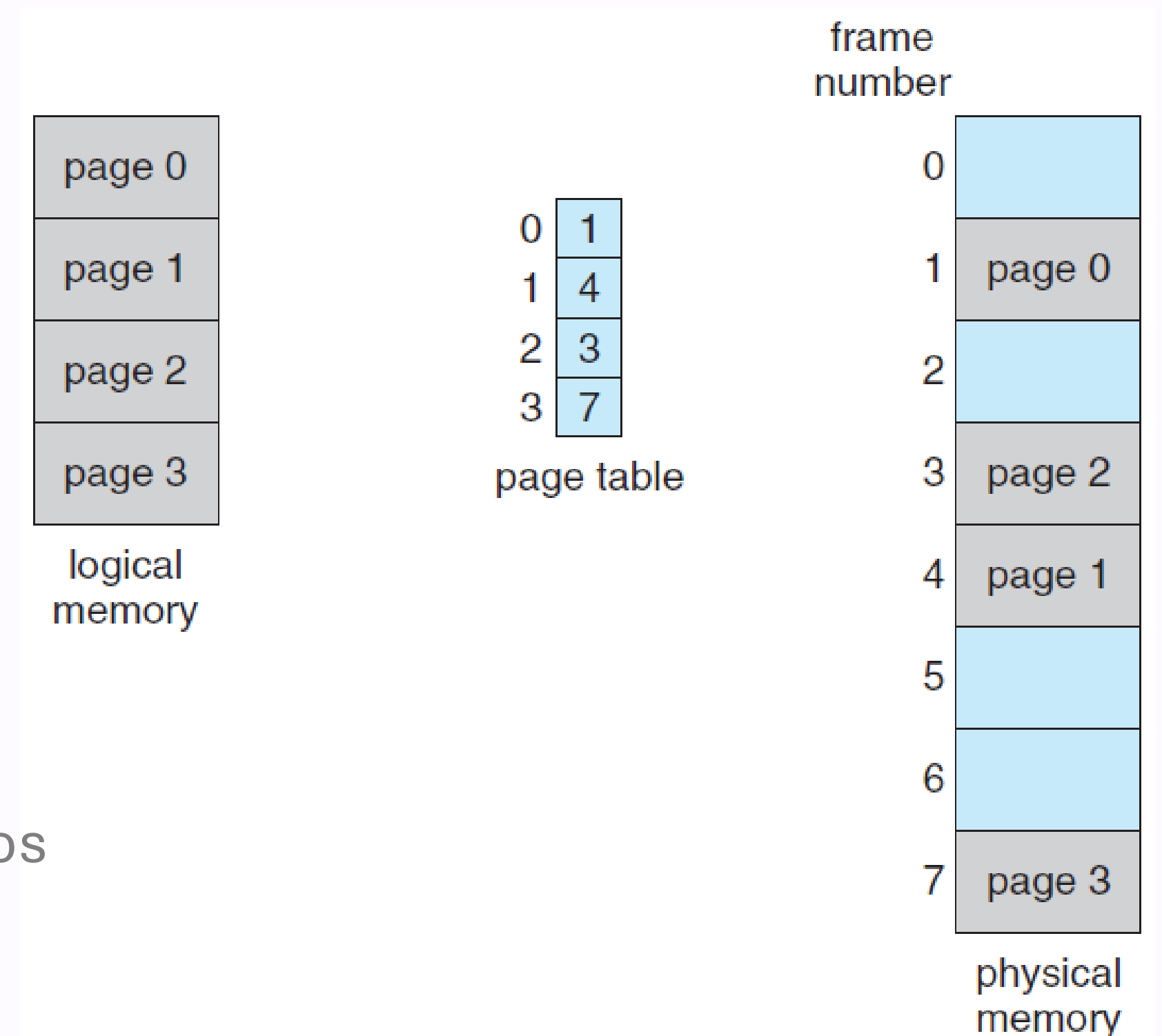
▪O espaço de endereçamento físico de um processo **não é contíguo**

▪**Método base**

- memória física dividida em blocos de **tamanho fixo** (frames)
  - tamanho é potência de 2 (facilita tradução; ver slide a seguir)
- memória lógica é dividida em blocos **do mesmo tamanho** (páginas)
- para correr um programa de tamanho igual a n páginas, encontrar n frames disponíveis (onde quer que se encontrem)
- usar uma tabela de páginas **traduz** os endereços lógicos para endereços físicos

▪Ao usar a memória que está disponível, evita-se a fragmentação externa e a necessidade de compactação

- não se evita no entanto a fragmentação interna



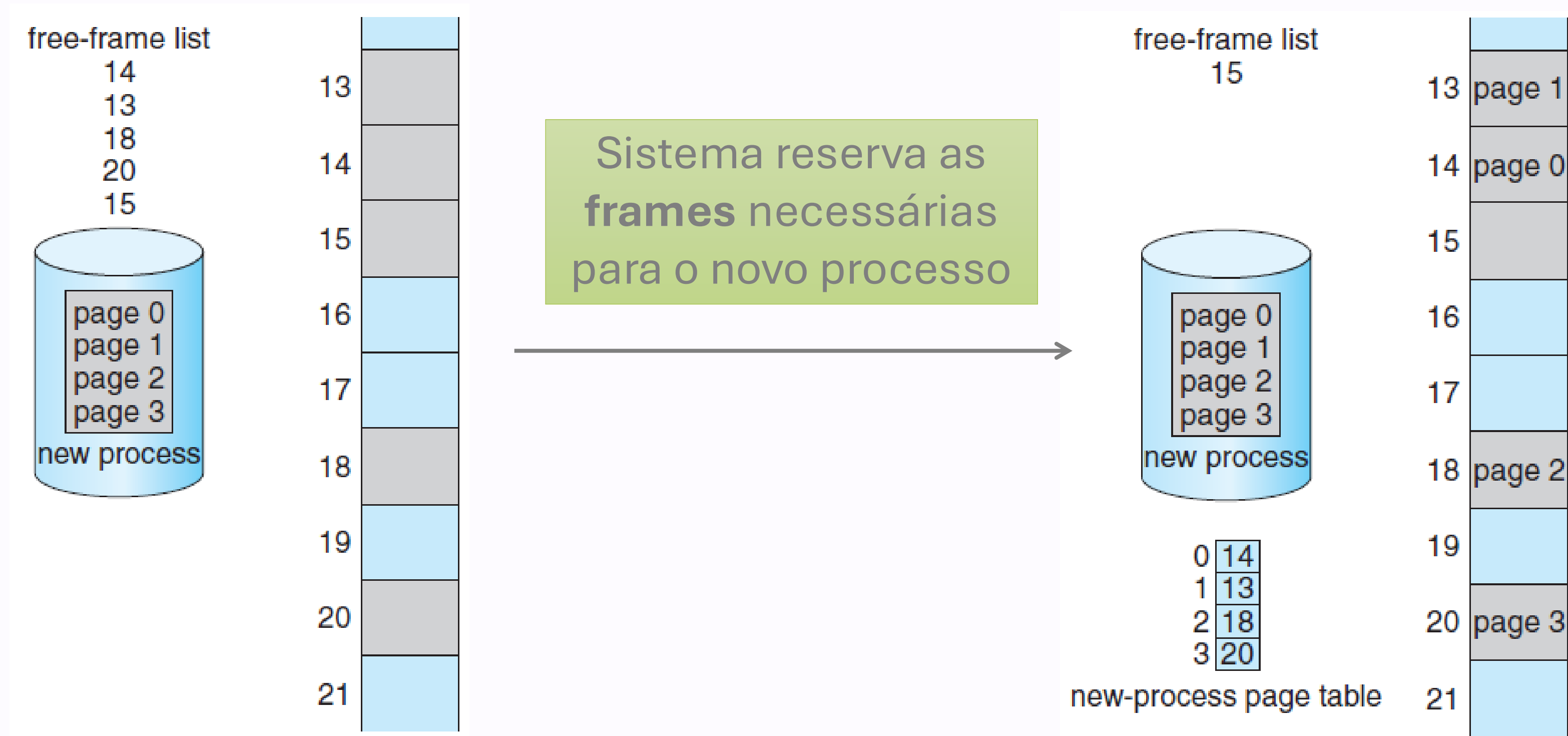


# PAGINAÇÃO - PAGE FAULT

## **Todos os processadores atuais (desktop e servidor) utilizam TLB**

- Quando um processo tenta aceder a uma página e essa página não está em memória ocorre uma page fault
  - A MMU gera uma exceção, que é processada pelo page fault handler:
    - O page fault handler consulta uma outra estrutura de dados (address map do processo) para determinar:
      - se a referência é válida: protecção;
      - e, em caso afirmativo, onde se encontra a página.
    - O handler obtém uma frame livre.
    - Transfere a página do disco para a frame.
    - Atualiza a page table.
- A CPU reinicia a execução da instrução que causou a page fault:
  - mais fácil dizer do que fazer, mas tem a ver com o Hardware

# RESERVA DE FRAMES



## ▪Em suma:

- o utilizador vê a memória como um **espaço único**, contíguo, mas na realidade o programa está **“espalhado”** pela memória física
- hardware de tradução de endereços **“reconcilia”** a memória lógica e a física

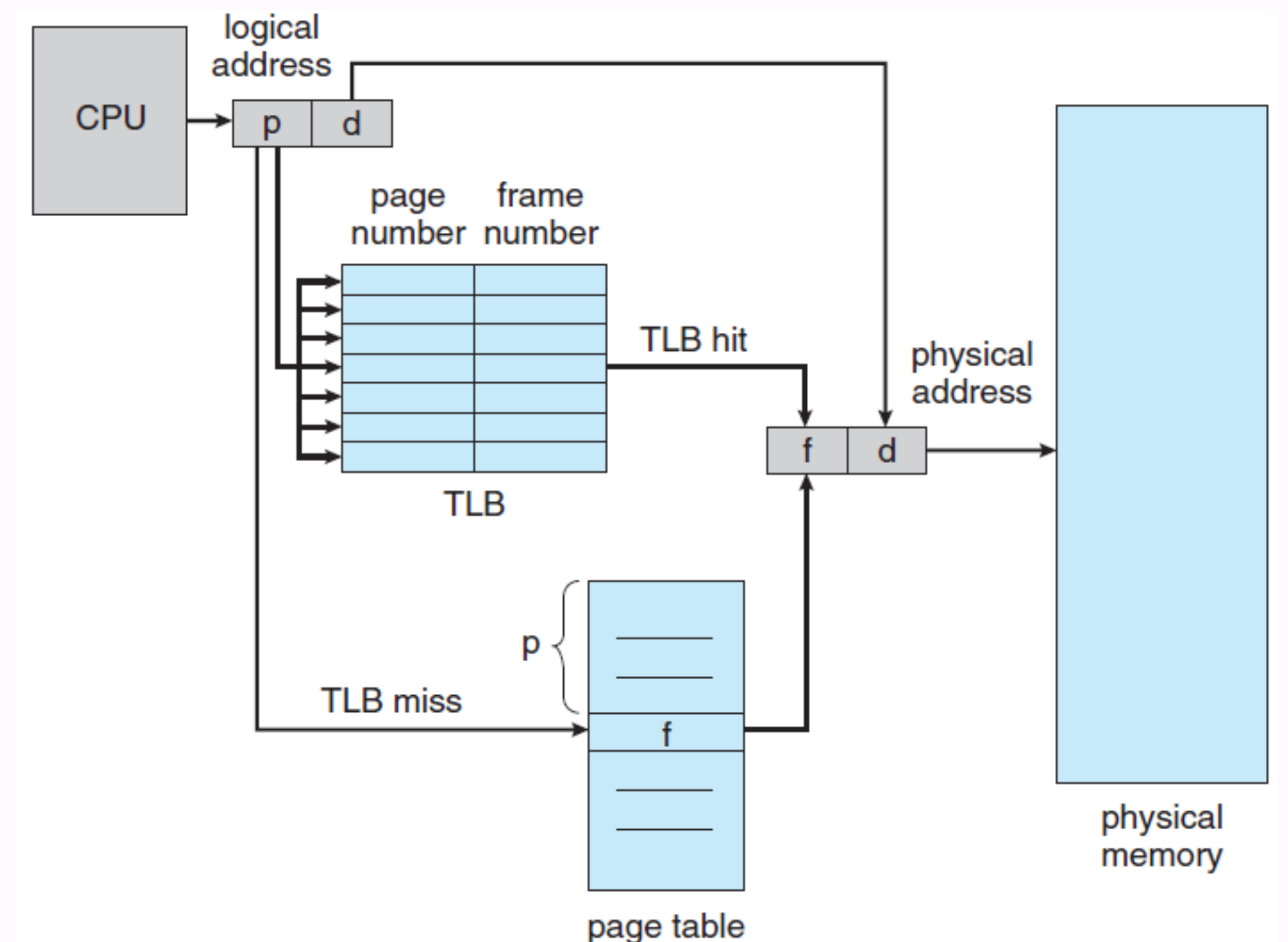
# IMPLEMENTAÇÃO

- Um método simples seria manter as tabelas de páginas **em memória** (uma por processo), sendo necessários dois registros

- PTBR (Page-table base register), **apontador** para a tabela de páginas
- PRLR (Page-table length register), com o **tamanho** da tabela
- cada acesso aos dados ou instruções requer **dois** acessos à memória (um para a tabela de paginação; outro para os dados/instrução), logo acesso à memória é **duas vezes** mais lento!

- **Possível solução?**

- usar uma **cache** (em hardware) que permita buscas rápidas: um **translation look-aside buffer (TLB)**
- a cache é **pequena** por isso só se guardam algumas entradas da tabela
- quando se gera um endereço lógico procura-se **primeiro no TLB**, o que é muito rápido
  - é memória associativa, permitindo comparação **simultânea** entre todas as entradas
  - o TLB lookup faz parte do pipeline de instruções, não havendo por isso qualquer redução de desempenho
- Se não estiver na TLB (TLB miss) tem de se procurar na tabela que está em memória

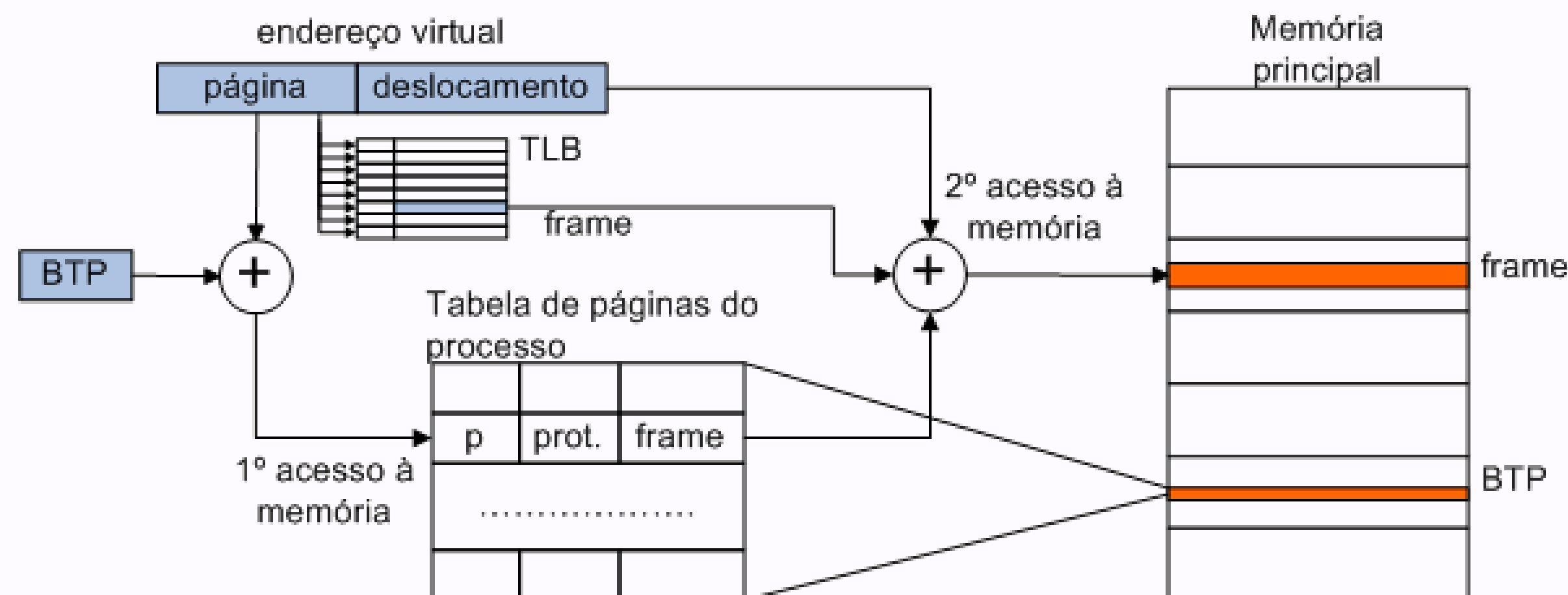




# PAGINAÇÃO

## ■ Recurso a memória TLB - Translation Lookaside buffer

- TLB - é uma cache da CPU usada para acelerar o processo de tradução dos endereços virtuais
- Basicamente, mapeia endereços virtuais em endereços físicos
- A memória TLB permite que cada um dos seus elementos seja pesquisado em simultâneo. Dado um valor da página, este é comparado, em simultâneo, com todos os elementos da tabela, devolvendo, em caso de sucesso, o endereço base da referida página (frame).
- A chave de pesquisa é o endereço virtual e o resultado da pesquisa é o endereço físico
- Se o endereço não existir na TLB, a tradução usa a tabela de páginas (mais lenta no acesso) – nalguns sistemas pode estar na memória secundária, o que a torna ainda mais lenta

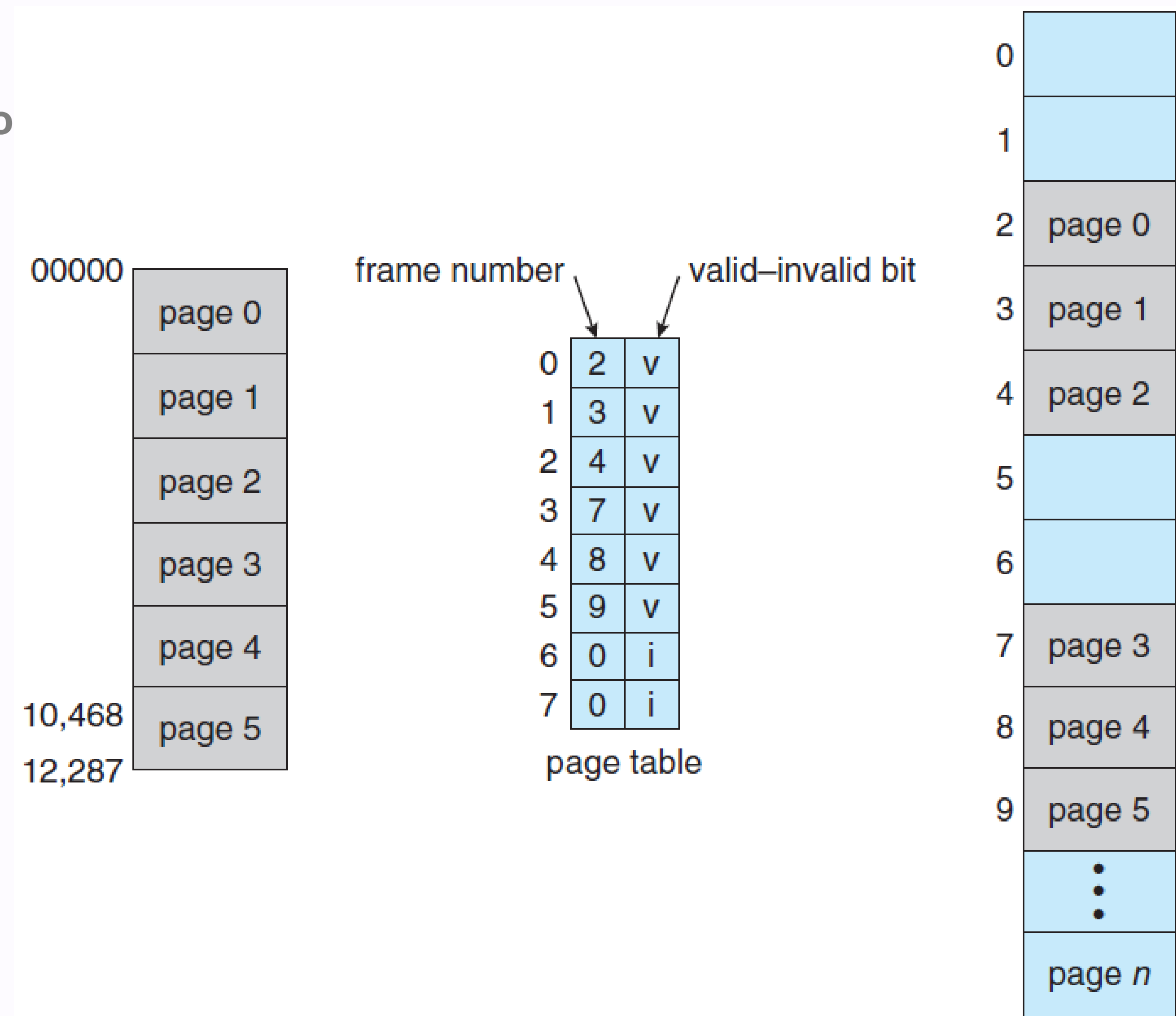


# SWAPPING

- **Processo retirado **temporariamente** da memória (para disco, tipicamente)**
  - ocorre quando a carga do sistema é elevada e começa a faltar memória
- **Dependendo de como os endereços foram atribuídos, quando o processo volta a ser colocado em memória pode ir para a mesma zona ou para outra**
  - se atribuição de endereços é feita em tempo de compilação ou de carregamento tem de ser para a **mesma** zona de memória
  - se é feita em tempo de execução pode ser para local **diferente**
- **Em sistemas móveis normalmente não se usa swapping por **limitações** de espaço em disco (e porque é limitado o número de escritas em memória flash)**
  - quando começa a faltar memória, sistemas como o iOS ou o Android pedem às aplicações para libertarem memória
  - se não libertarem a memória suficiente, correm o risco de serem **terminadas** pelo SO

# PROTEÇÃO

- A proteção de memória é implementada associando **bits de proteção** a cada frame
  - bits *read-only vs read-write vs execute-only*
  - bit *valid-invalid*
    - página associada é **válida**, i.e., pertence ao espaço de endereçamento lógico do processo





# SWAPPING

- **Processo retirado **temporariamente** da memória (para disco, tipicamente)**
  - ocorre quando a carga do sistema é elevada e começa a faltar memória
- **Dependendo de como os endereços foram atribuídos, quando o processo volta a ser colocado em memória pode ir para a mesma zona ou para outra**
  - se atribuição de endereços é feita em tempo de compilação ou de carregamento tem de ser para a **mesma** zona de memória
  - se é feita em tempo de execução pode ser para local **diferente**
- **Em sistemas móveis normalmente não se usa swapping por **limitações** de espaço em disco (e porque é limitado o número de escritas em memória flash)**
  - quando começa a faltar memória, sistemas como o iOS ou o Android pedem às aplicações para libertarem memória
  - se não libertarem a memória suficiente, correm o risco de serem **terminadas** pelo SO

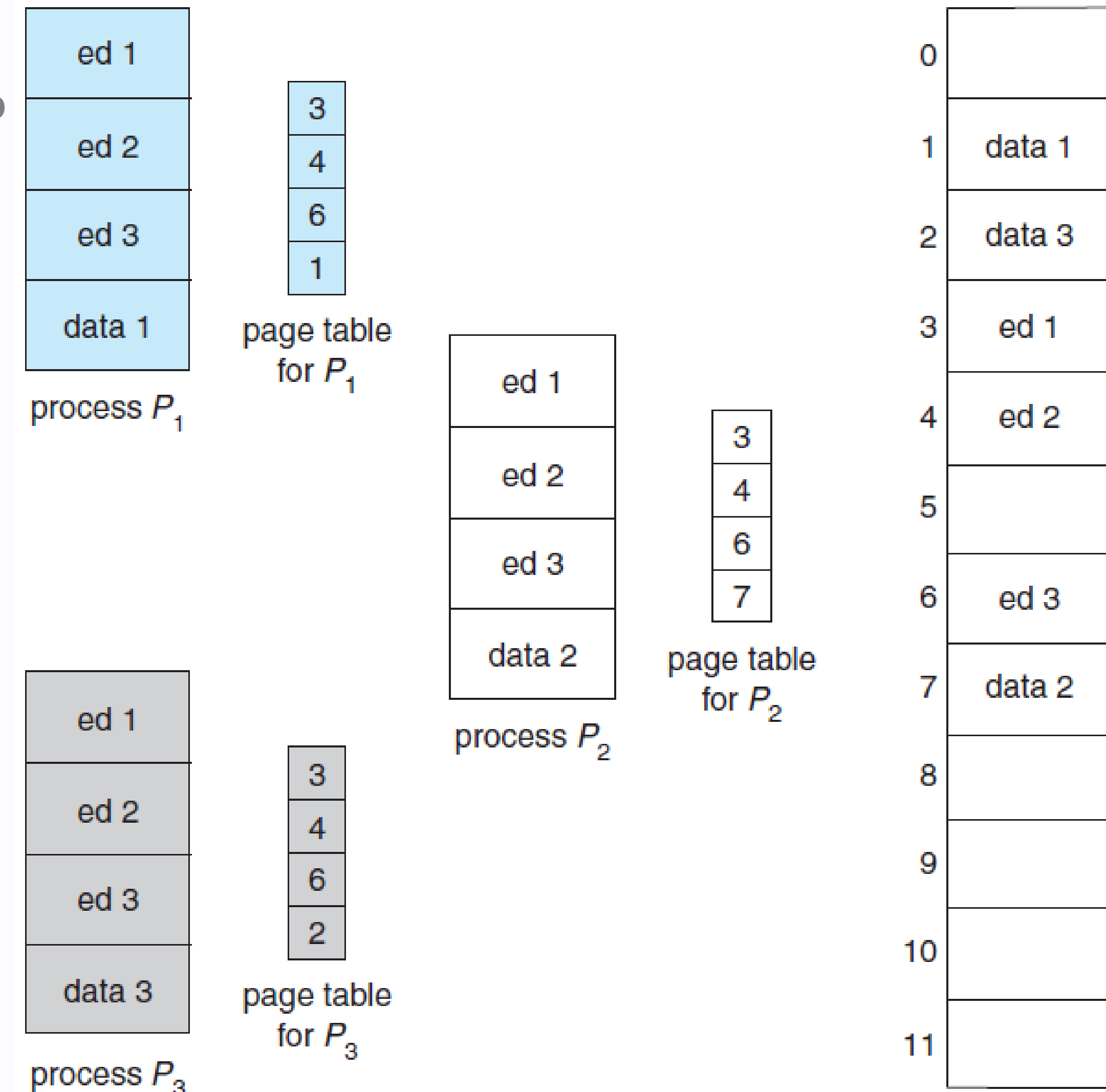
# PÁGINAS PARTILHADAS

## ▪ Código **partilhado**

- apenas necessário manter **uma cópia** de código reentrante (*read-only*) em memória, partilhada entre processos
  - no exemplo, um editor de texto, ed
- o código partilhado é associado às **mesmas** frames para todos os processos

## ▪ Código e dados **privados**

- cada processo mantém uma **cópia separada** desse código e dos dados
- as páginas desse código e dos dados são associadas a *frames* **diferentes**



# PROBLEMA

Os computadores atuais têm um espaço de endereçamento lógico muito grande ( $2^{32}$ ,  $2^{64}$ )

a tabela de páginas pode tornar-se **enorme!**

Exemplo: espaço de endereçamento de 32 bits ( $2^{32}$  endereços) e páginas de 4kB ( $2^{12}$  bytes)

Nº de páginas:  $2^{32}/2^{12} = 2^{20} > 1 \text{ milhão de entradas}$

se páginas fossem maiores, tínhamos menos entradas ...

... mas a fragmentação interna era maior

Assumindo que cada entrada da tabela ocupa 4 bytes, a tabela de cada processo ocupa  $4 \times 2^{20} = 4\text{MB}$

será **difícil** colocar a tabela de páginas numa zona de memória contígua...

E agora **experimentem** fazer as contas considerando um espaço de endereçamento de 64 bits...

Solução:

**dividir a tabela de páginas**



# TABELA DE PÁGINAS HIERÁRQUICA

- Uma técnica simples é fazer uma tabela de páginas de **dois níveis**

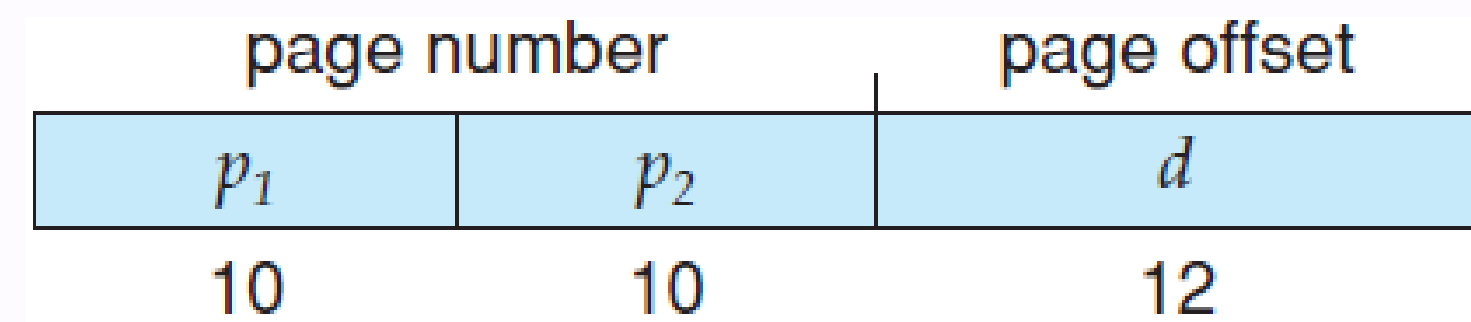
- De novo o exemplo anterior

- espaço de endereçamento lógico numa máquina de 32 bits com páginas de 4kB (212) pode ser dividido em:

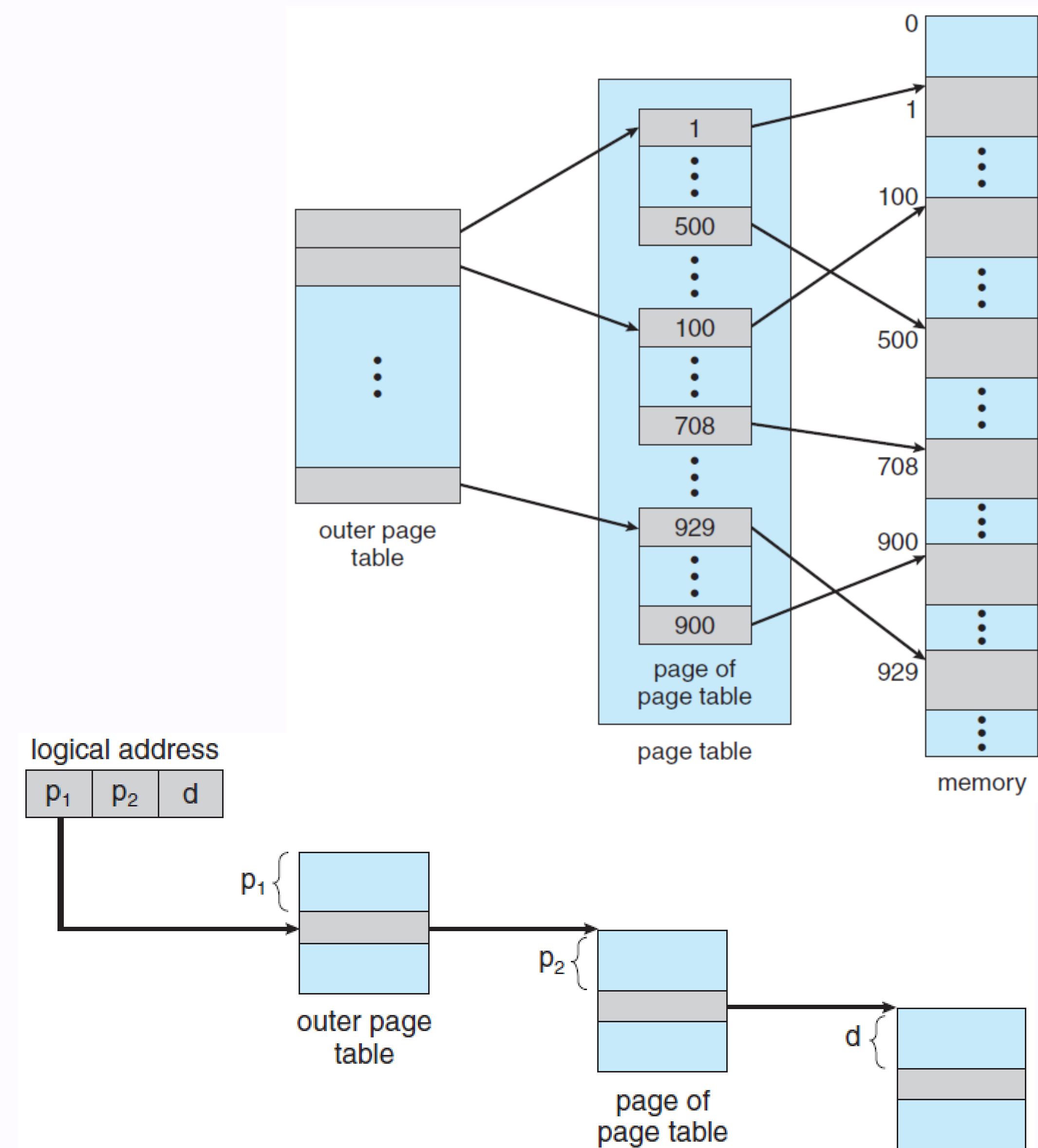
- número de página com 20 bits
    - offset com 12 bits (d)

- **Paginando** a própria tabela de páginas, o número de páginas é dividido em duas partes. Por exemplo:

- 10 bits - seleciona tabela interior (p1)
    - 10 bits - seleciona a frame (p2)



- $p_1$  é o **índice** para a tabela exterior e  $p_2$  é o **offset** da tabela interior



# TABELA DE PÁGINAS MULTI-NÍVEL

E se o espaço lógico de endereçamento for ainda maior (e.g., 64 bits)?

Dois níveis já não são suficientes

| outer page | inner page | offset |
|------------|------------|--------|
| $p_1$      | $p_2$      | $d$    |
| 42         | 10         | 12     |

Três níveis então?

| 2nd outer page | outer page | inner page | offset |
|----------------|------------|------------|--------|
| $p_1$          | $p_2$      | $p_3$      | $d$    |
| 32             | 10         | 10         | 12     |

Ou quatro? Talvez cinco? Ou mesmo seis? Ou...

Como se pode verificar, para arquiteturas de 64 bits a estrutura hierárquica começa a ser pouco apropriada

De qualquer forma a arquitetura x86-64 usa tabela de páginas multi-nível (usados apenas 48 bits; tabela com 4 níveis)

| unused | page map level 4 | page directory pointer table | page directory | page table | offset |
|--------|------------------|------------------------------|----------------|------------|--------|
| 63     | 48 47            | 39 38                        | 30 29          | 21 20      | 12 11  |
|        |                  |                              |                |            | 0      |

# TABELA DE PÁGINAS HASHED

Comum em espaços p ç de endereçamento **superiores** a 32 bits

A tabela de páginas passa a ser uma **tabela hash**

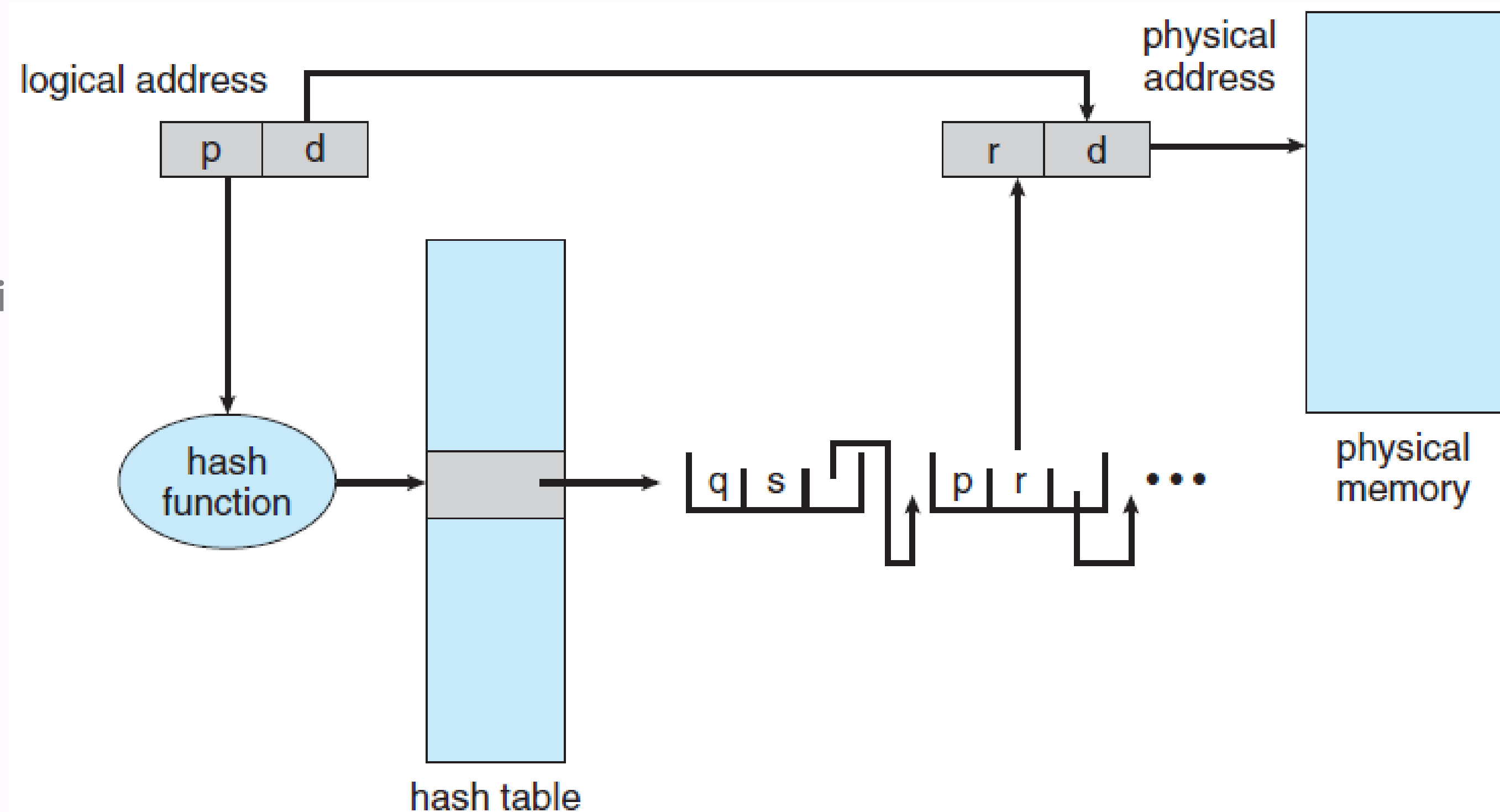
cada posição da tabela hash vai conter um ponteiro para uma lista ligada

a lista ligada contém **todos** os elementos cuja hash calha na mesma posição, resolvendo desta forma as **colisões**

cada elemento da lista contém o número de página, a frame respectiva e um ponteiro para o próximo elemento

## Algoritmo:

corre-se a **função hash** com o número de página (p)  
resultado é o **índice** da tabela onde se vai procurar p  
procura-se p na lista ligada e extrai-se a frame respectiva (r).





# TABELA DE PÁGINAS INVERTIDA

Normalmente, **cada** processo tem uma tabela de páginas associada

cada uma dessas tabelas pode ser enorme, levando as tabelas de páginas a consumir **imensa** memória física

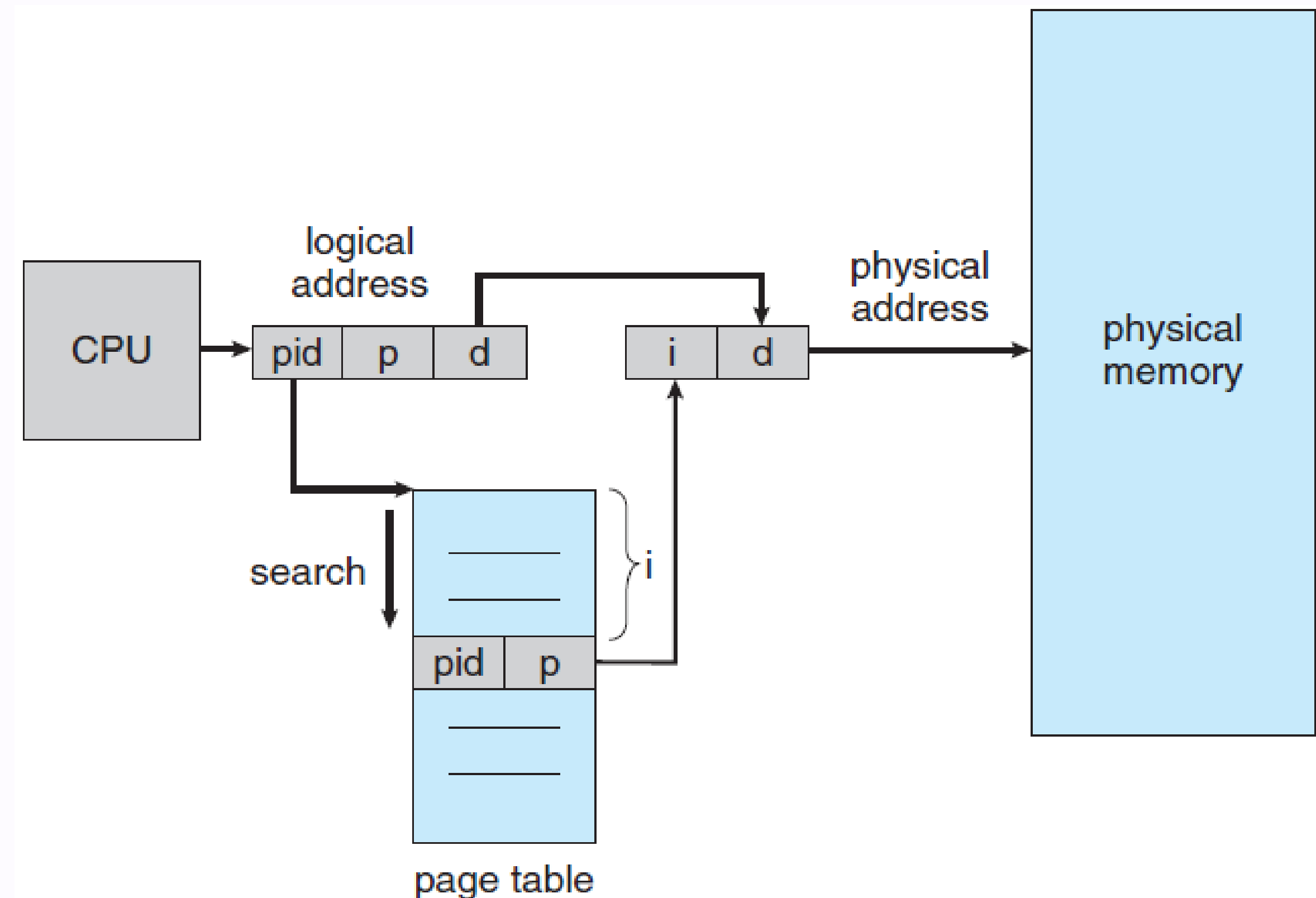
Com uma tabela de páginas invertida há **apenas uma** tabela de páginas com **apenas uma** entrada por frame em memória

cada entrada nessa tabela tem o endereço virtual da página (p) e o identificador do processo “dono” dessa página (pid)

a entrada é armazenada na localização **real** de memória (i)

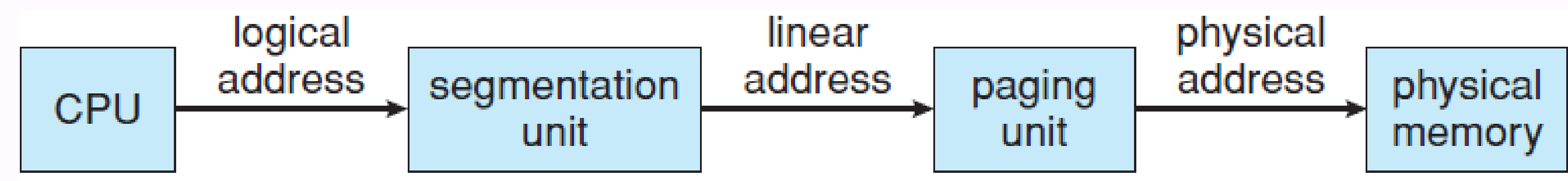
Desta forma **reduz-se** a memória necessária para paginação, mas **aumenta-se** o tempo de procura

para mitigar este problema pode usar-se uma tabela hash como tabela invertida

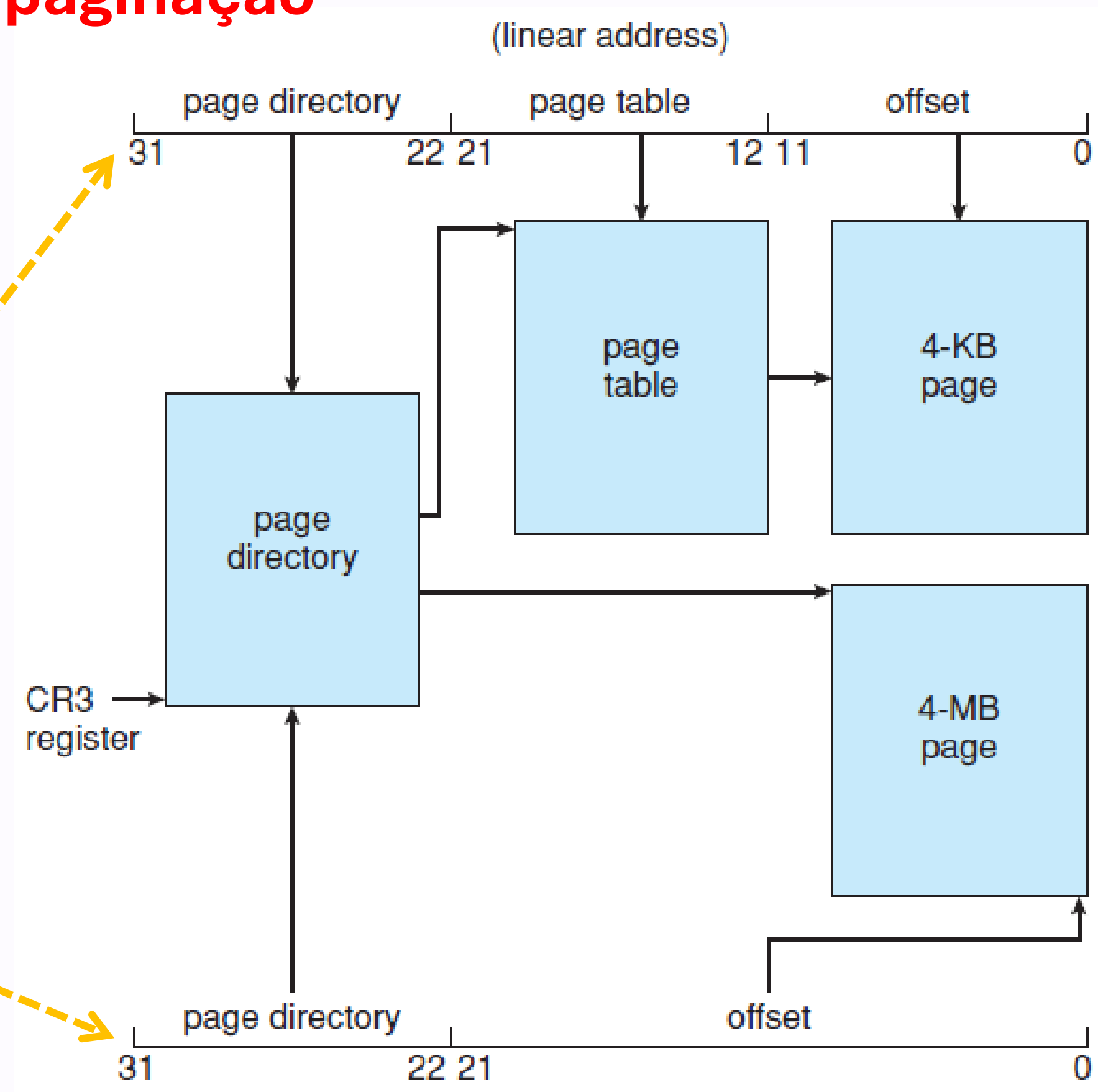
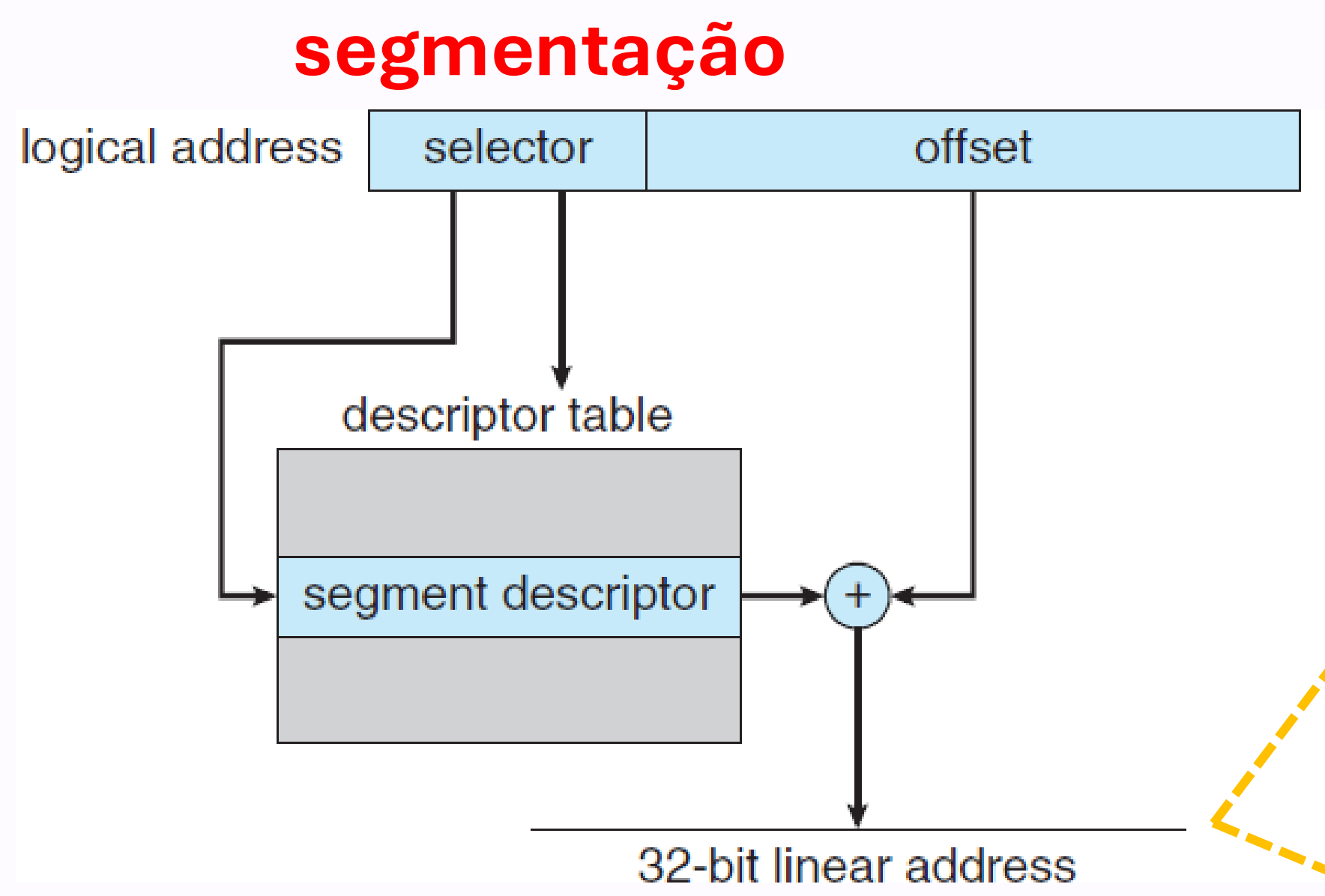


# INTEL PENTIUM

Suporta **segmentação com paginação**



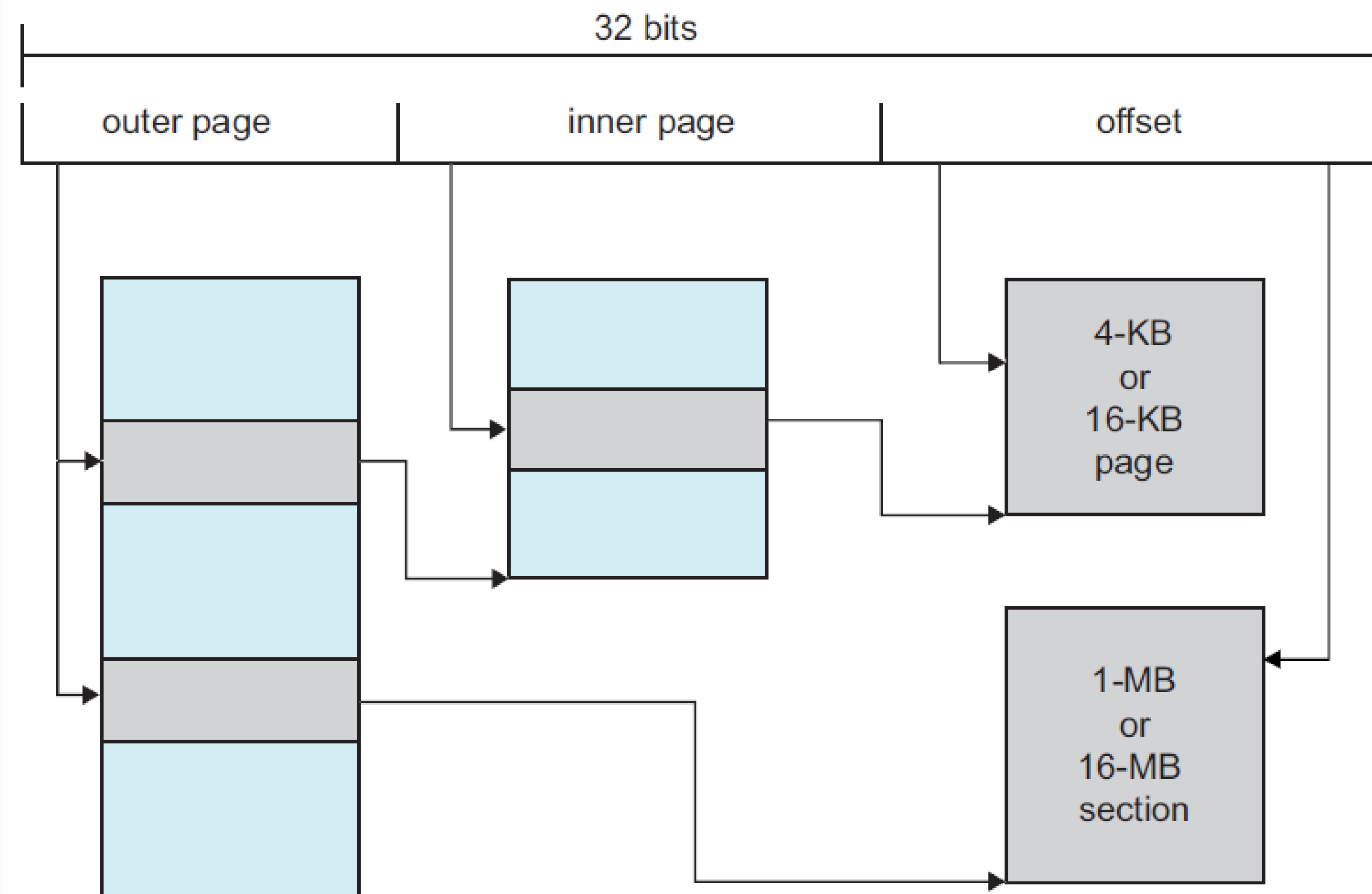
**paginação**



# ARM

Os chips da Intel dominaram o mercado dos computadores nos últimos 30 anos, **mas 99% dos dispositivos móveis (smartphones, tablets) usa processadores ARM**

A arquitetura ARM usa **paginação de 2 níveis** para páginas pequenas (4KB e 16KB) e **paginação de 1 nível** para páginas grandes (1MB e 16MB)





---

QUESTÕES?